

A detailed, colorful micrograph of a semiconductor chip, showing intricate circuit patterns in shades of red, green, blue, and yellow. A large, semi-transparent blue rectangle is overlaid on the left side of the image, containing the title text.

Marco Ferroni's Portfolio

Profile

I'm a 23 years old passionate student from a small town near Ravenna (IT).

- **Education**

- **Master Degree (PRESENT):** Electrical Engineering at ETH Zurich (current GPA 5.55/6).
- **Bachelor:** Electronics Engineering at Politecnico di Milano (110L/110 with 29.48/30 final GPA).

- **Scholarships**

- **Best freshmen award:** 1000 euros scholarship awarded to the best freshmen of Politecnico di Milano (top 2.5%).
- **Fee exemption:** in my Bachelor due to my high GPA ($>29/30$) I was granted a fee exemption.

- **Job experience**

- **Circuit Design Intern at SynSense:** currently employed for six months as part of the IC design team, I am learning the full stack of asynchronous IC design.
- **Academic guest at EMPA:** I've performed quantum simulations on CNTs

ETH zürich

Portfolio



Me



Me and my cat!



Design of an Asynchronous Tree NoC

Design of a quad-tree NoC -Overview

- **Context**

- **Master's thesis project:** pursued at Stanford University under the supervision of Prof. Kwabena Boahen.
- **Scope:** Implement a quad-tree asynchronous NoC with multicasting functionality for an LLM knowledge-retrieval ASIC.

- **Goal**

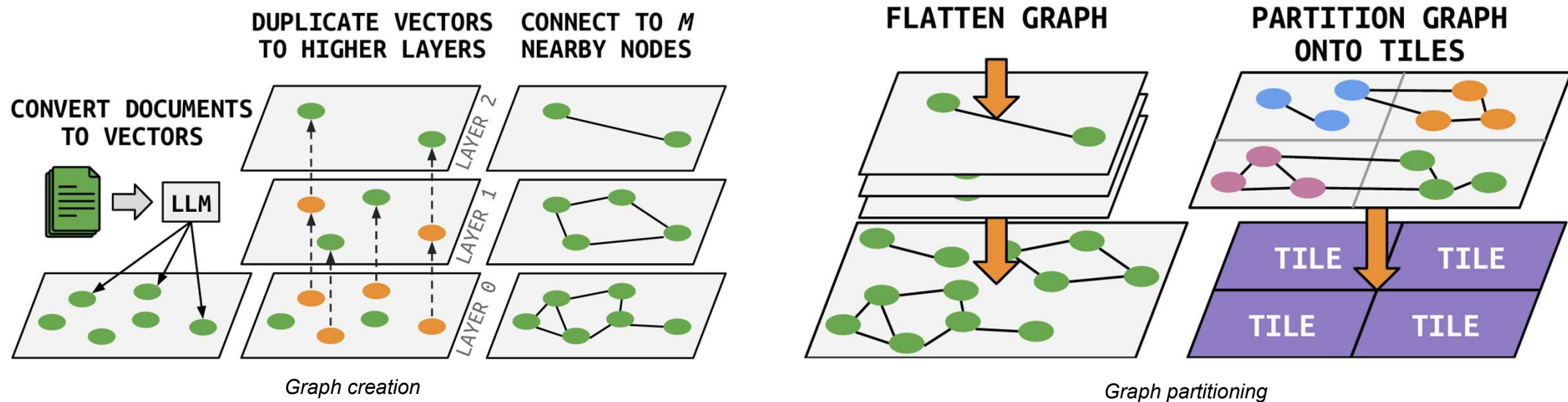
- Go from a Python model to silicon

- **Workflow**

- **Architecture exploration:** test ideas using a Python model
- **CHP writing:** use CHP (Communicating Hardware Processes), a high-level description language designed for asynchronous circuits.
- **Decomposition:** break the CHP design down into smaller, simpler blocks.
- **PRS writing:** describe the design at the logic level using PRS, a lower-level abstraction.
- **Layout and GDS:** finally as last step I will produce the layout and gds

Design of a quad-tree NoC –HAMSA chip

The NoC will be integrated into a chip called HAMSA, a fully asynchronous ASIC designed to accelerate the HNSW (Hierarchical Navigable Small World) algorithm for approximate nearest-neighbor search. A graph is first constructed from the high-dimensional embedding space containing all embeddings in the dataset. The graph is then flattened and partitioned across multiple memory tiles to minimize inter-tile connections and communication overhead. During inference, the accelerator traverses the distributed graph to retrieve the embedding vector that best matches a given input query.



Design of a quad-tree NoC –HAMSA chip

- **Structure**

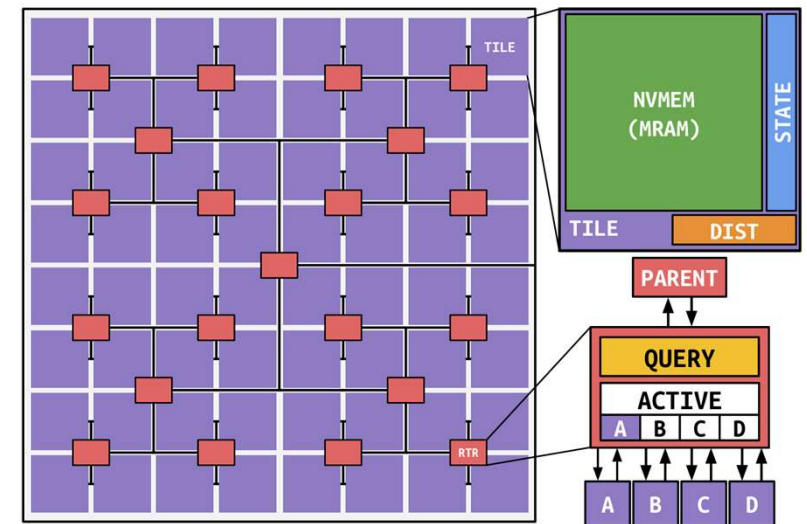
- **Fabric of tiles**

- Memory: stores the embeddings,
 - Compute unit: calculates the distance between embeddings and the input query
 - Control unit: manages execution

- **Quad tree NoC:** connects all the tiles together and allows to cache the query at every router node to reduce traffic

- **Communication**

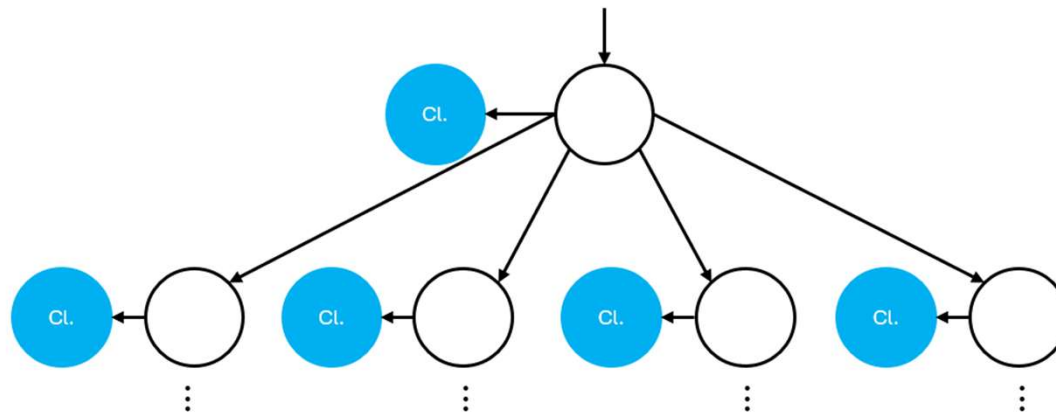
- **Sparse:** only a few tiles are actually used at a time
 - **Local:** the active tiles are usually close to each other thanks to the partitioning that reduces edges.



Hamsa chip structure

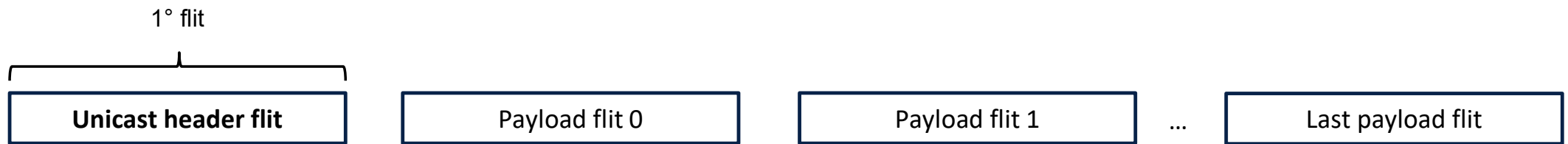
Design of a quad-tree NoC –NoC features

- **Unicasting:** support for point-to-point communication between any pair of tiles
- **Multicasting:** support for transmitting a message from one tile to multiple destinations
 - **Common payload:** the same payload can be delivered to all destinations within a multicast transmission
 - **Specific payload:** different payloads can be delivered to different destinations without additional overhead by leveraging the routing scheme itself
- **Query caching:** since queries are large, each router node includes a client that caches the query, reducing query movement and overall network traffic.



Tree NoC structure

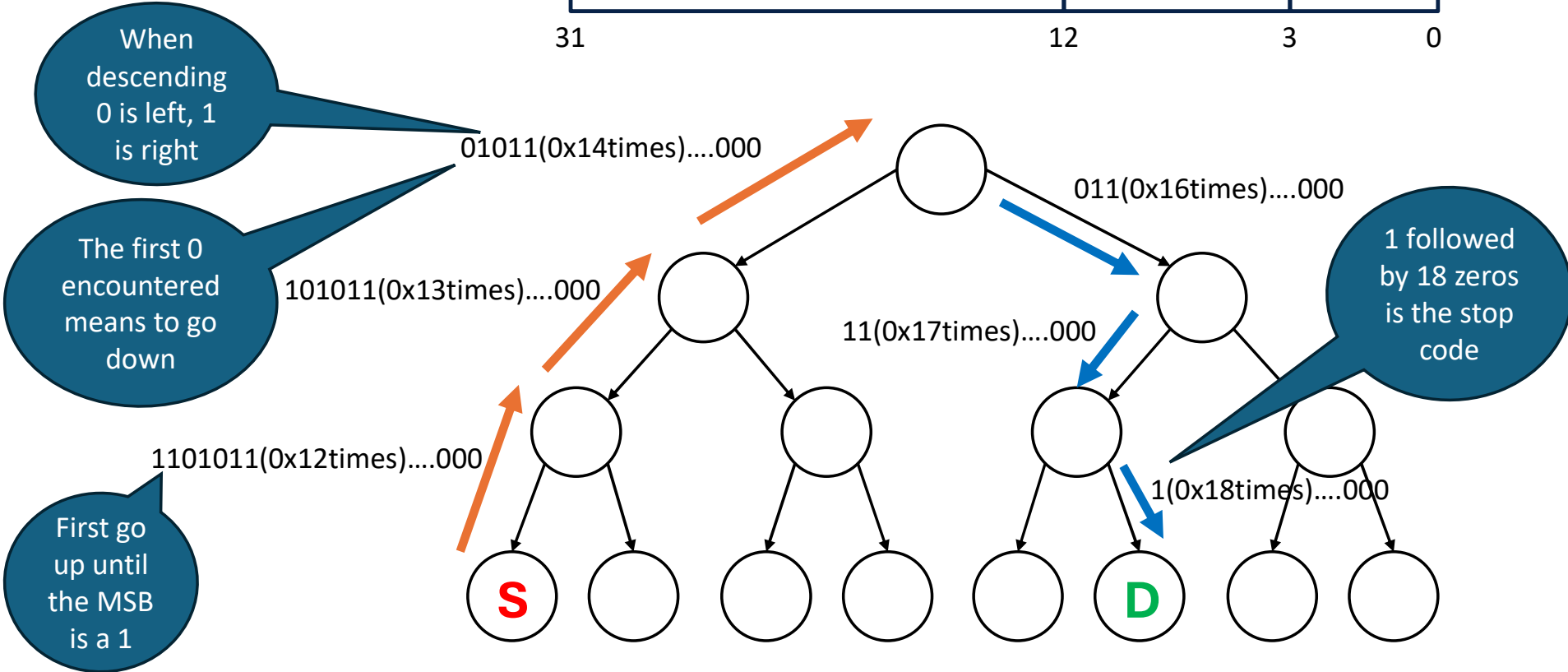
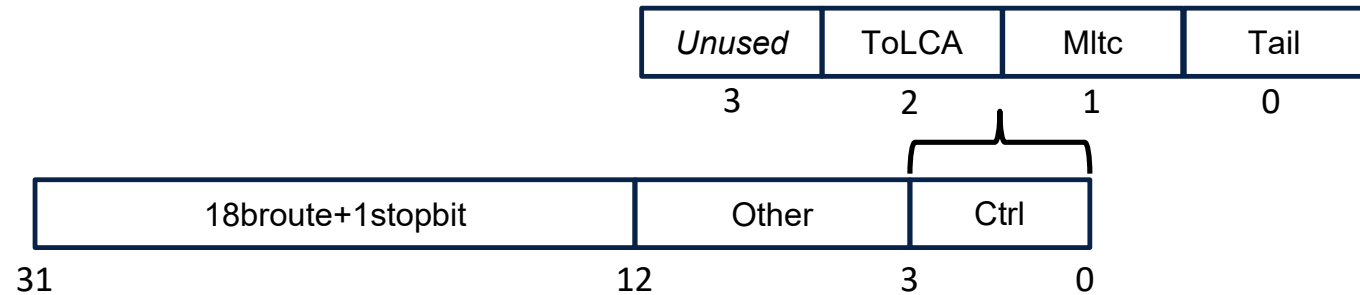
Design of a quad-tree NoC –Unicast Transmission



Design of a quad-tree NoC – Unicasting

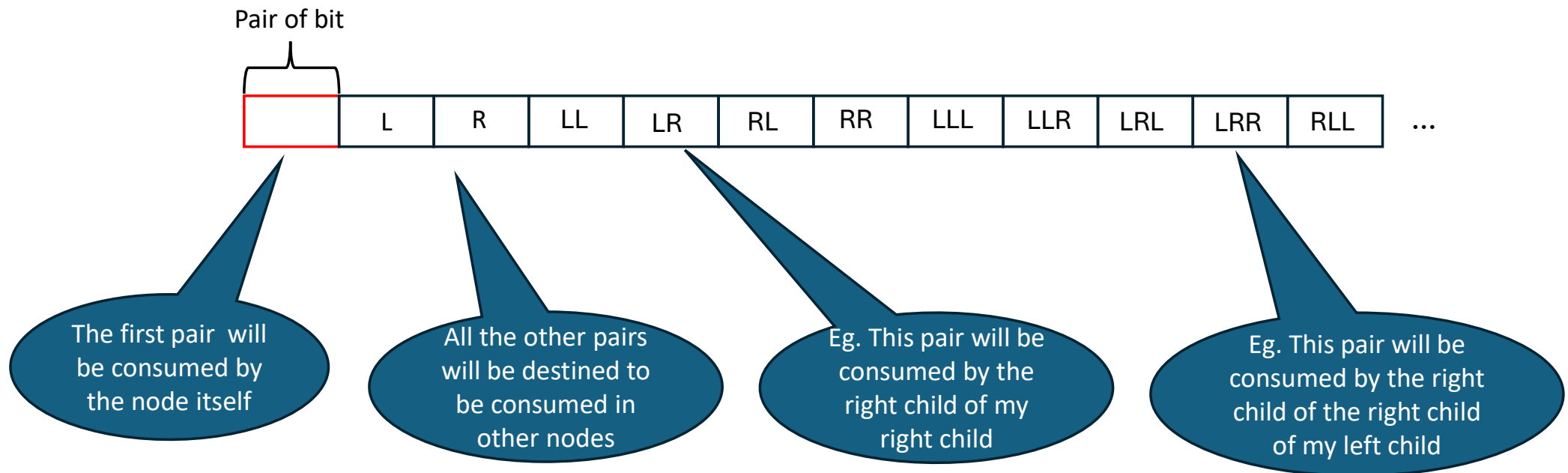
- BW=32 bits
- Must avoid deadlocks
- **Binary tree for simplicity**

Unicast header structure:



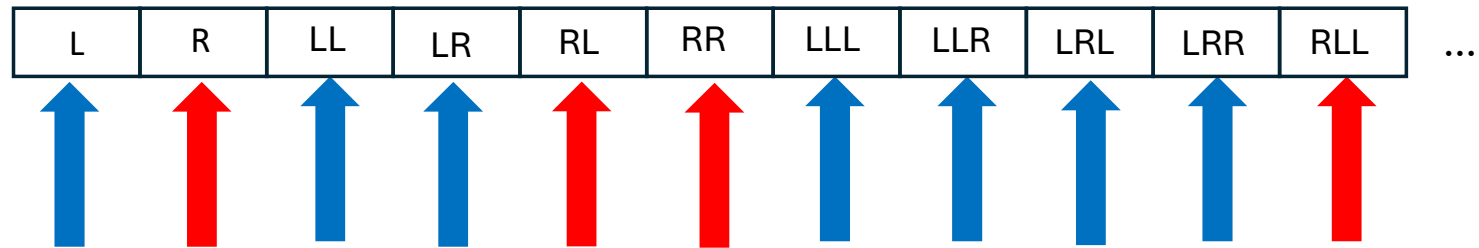
Design of a quad-tree NoC –Multicasting Route Encoding

For a binary tree, 2 bits are required for each node in the subtree that participates in routing the message. The MSB corresponds to the left child, while the LSB corresponds to the right child. If a bit is asserted, the corresponding child needs to receive part of the route. Again, in our case each node has actually 4 children, but for the sake of simplicity now we are assuming to have 2.



Design of a quad-tree NoC –Multicasting Route Encoding

When the node receives this route, it needs to parse it and send it to the children

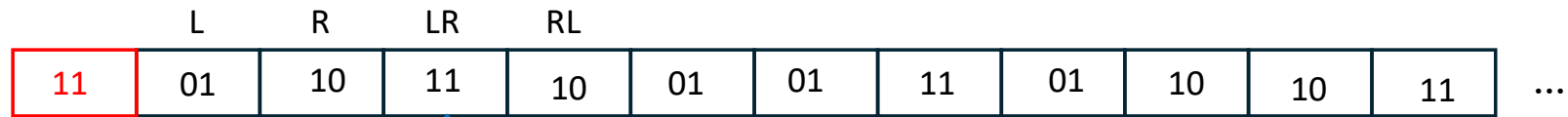


All the blues pairs will be appended one after the other to create the subroute for the left child

All the red pairs will be appended one after the other to create the subroute for the right child

Design of a quad-tree NoC –Multicasting Parsing Example

However the message will not always go through every node of the subtree, so in a real route many pairs will be missing



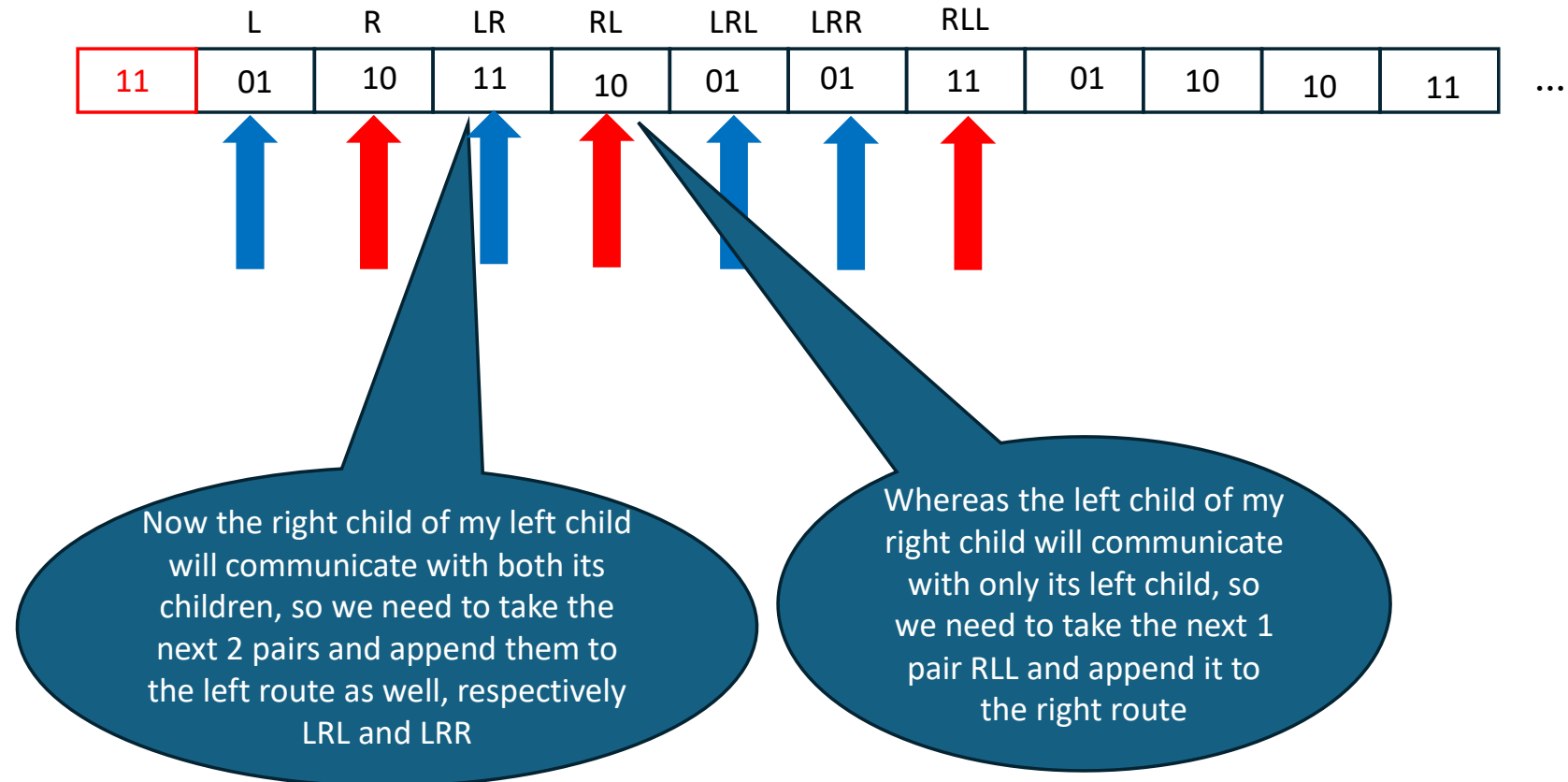
The message will go through both of my children; so I need to prepare two subroutes to send them

The left child will have to communicate with its right child and the right child with its left one

So the next two pairs in the route are respectively LR and RL

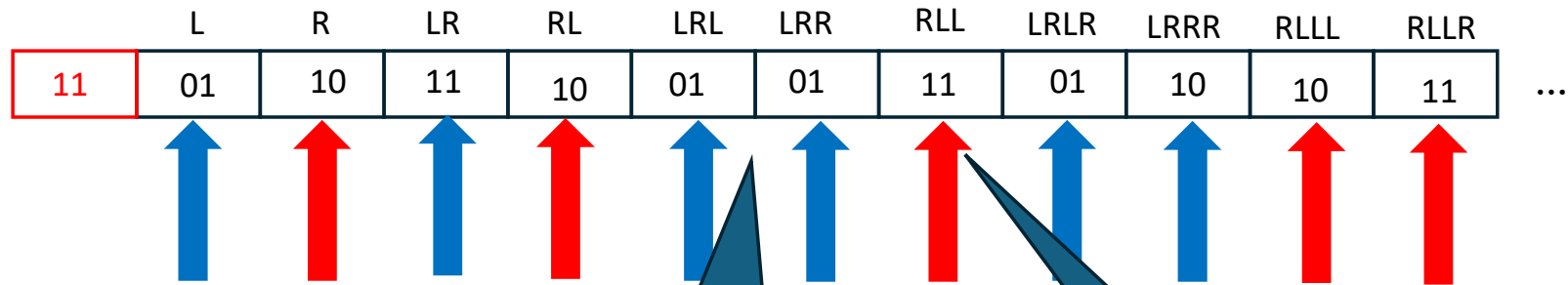
Design of a quad-tree NoC –Multicasting Parsing Example

However the message will not always go through every node of the subtree, so in a real route many pairs will be missing



Design of a quad-tree NoC –Multicasting Parsing Example

However the message will not always go through every node of the subtree, so in a real route many pairs will be missing

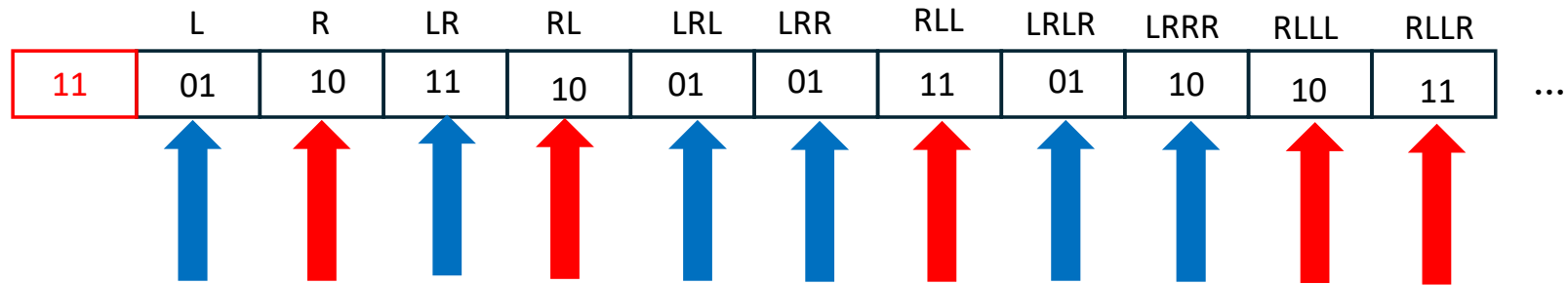


The left and right children of the right child of my left child will communicate with both their right child, so we need to take the next 2 pairs and send them on the left route as well, respectively LRLR and LRRR

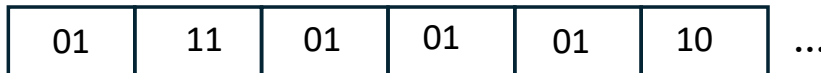
While the left child of the left child of my right child will communicate with both its children RLLL and RLLR

Design of a quad-tree NoC –Multicasting Parsing Example

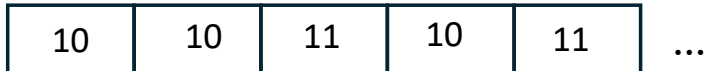
However the message will not always go through every node of the subtree, so in a real route many pairs will be missing



Left subroute



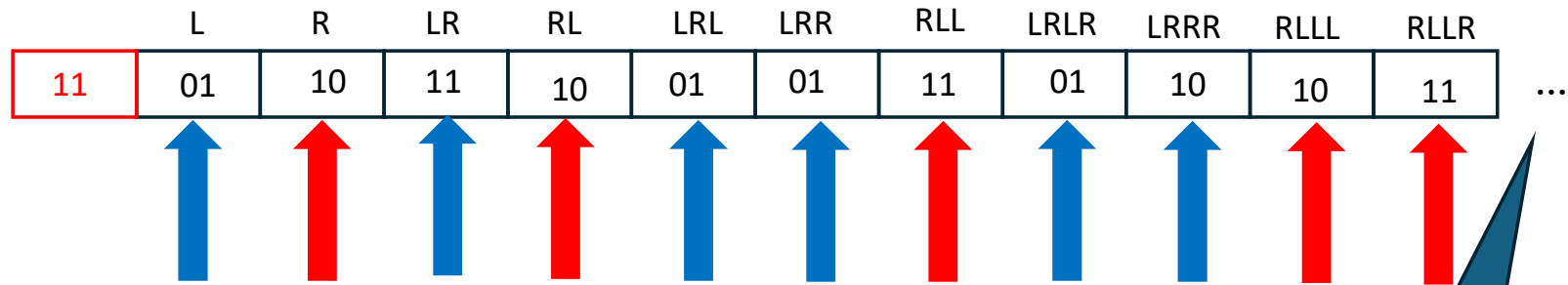
Right subroute



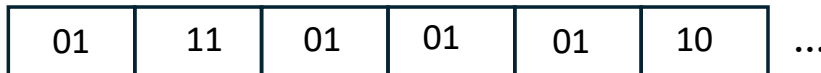
Once the filling of these buffers reach BW-4 (we reserve the 4 LSB for ctrl) we send them to the corresponding child and we flush them

Design of a quad-tree NoC –Multicasting Parsing Example

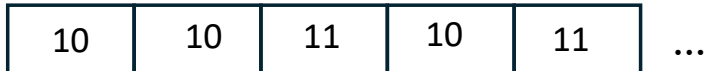
However the message will not always go through every node of the subtree, so in a real route many pairs will be missing



Left subroute



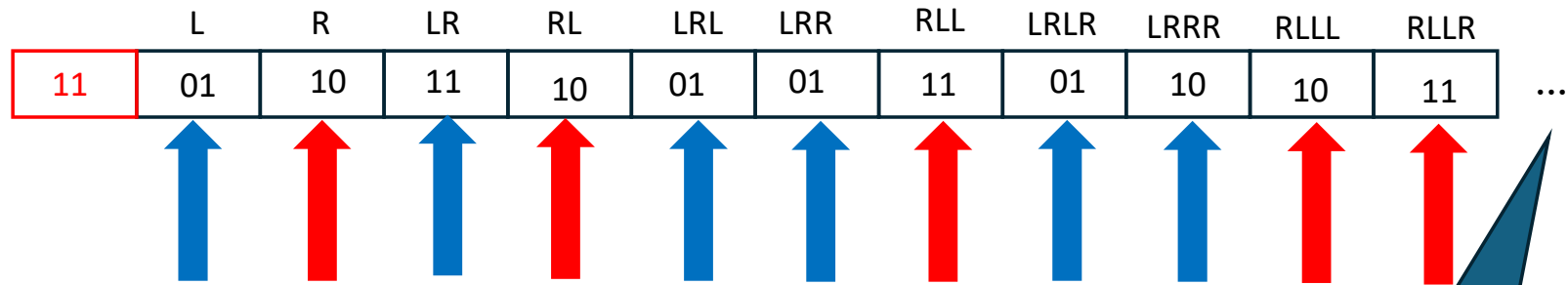
Right subroute



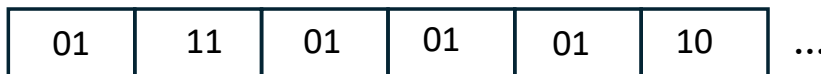
If we are done with the current route flit, we flush it, take a new one and then keep appending to the two subroute buffers

Design of a quad-tree NoC –Multicasting Parsing Example

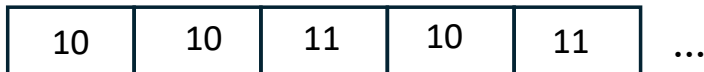
However the message will not always go through every node of the subtree, so in a real route many pairs will be missing



Left subroute

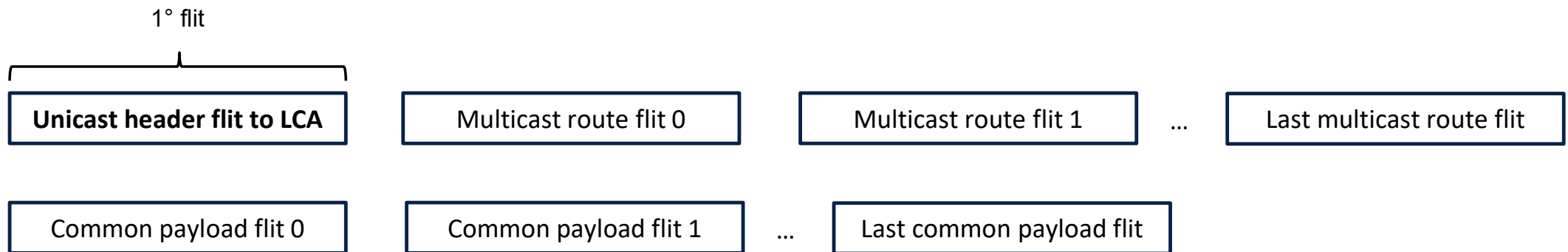


Right subroute



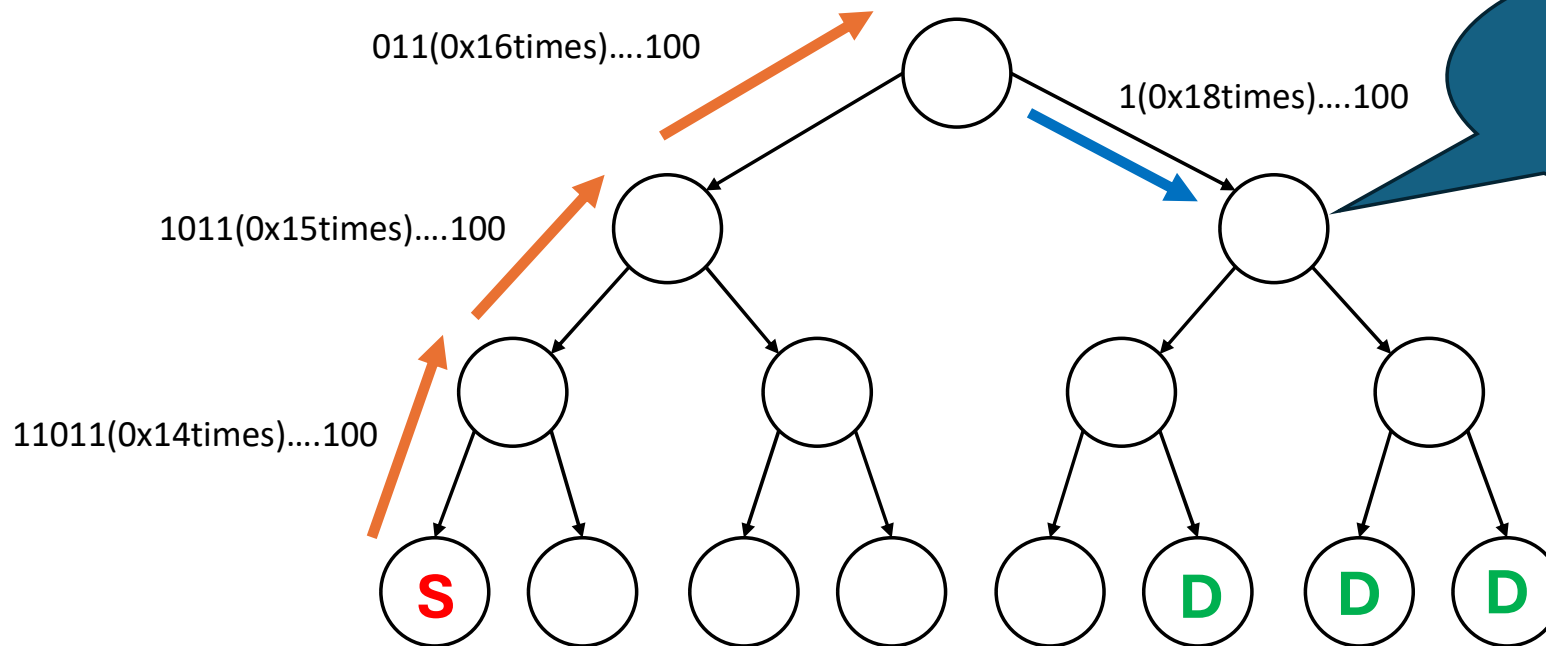
The parsing process stops when we first see a pair equal to 00, namely the stop code. The buffers are sent to the children and every next flit is common payload

Design of a quad-tree NoC –Multicast Transmission



Design of a quad-tree NoC –Multicasting

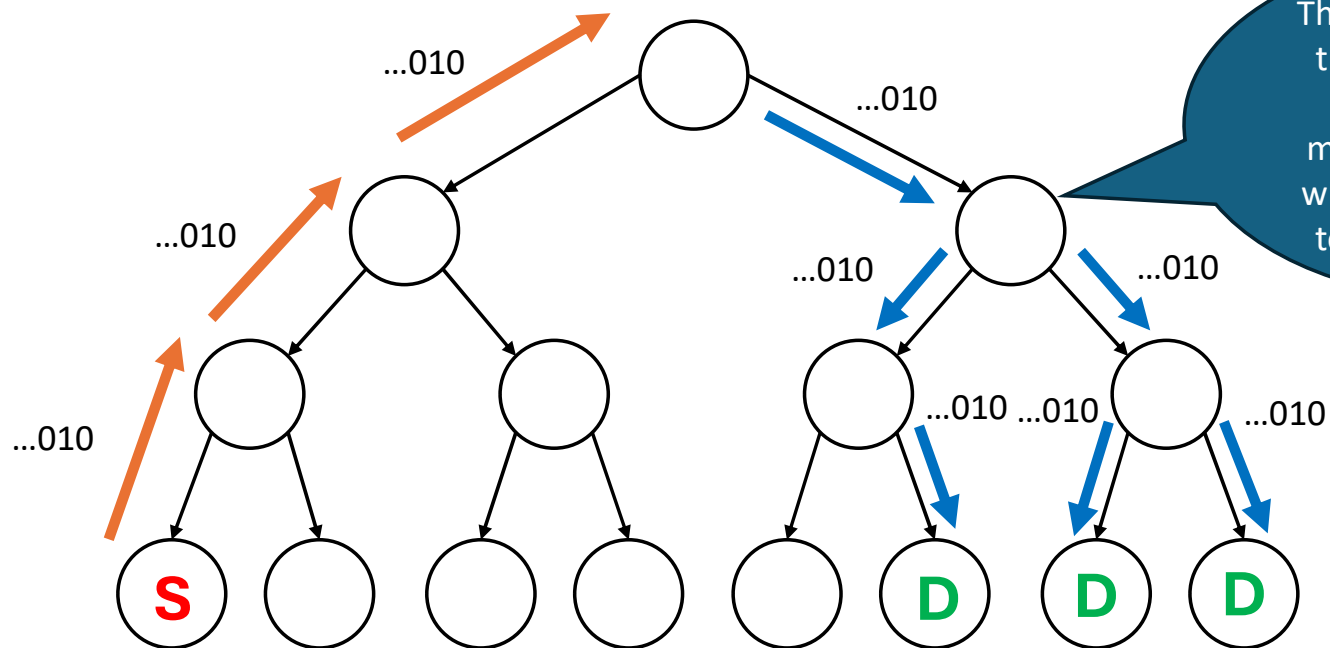
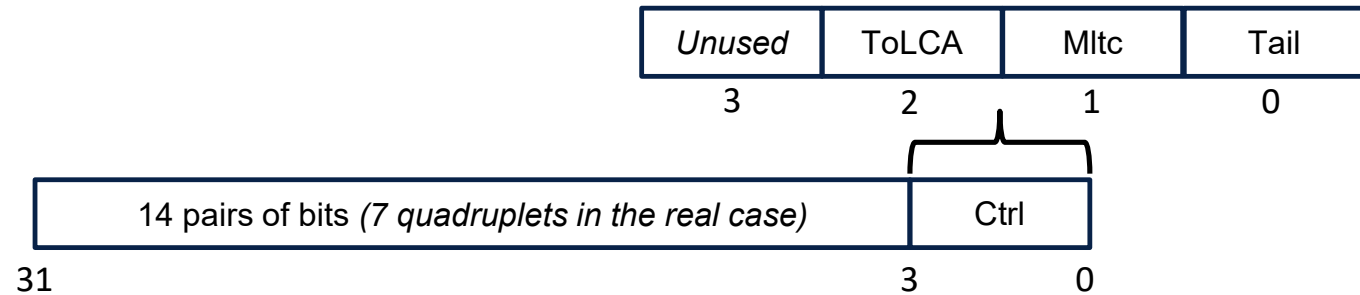
- BW=32 bits
- Must avoid deadlocks
- **Binary tree for simplicity**



Design of a quad-tree NoC –Multicasting

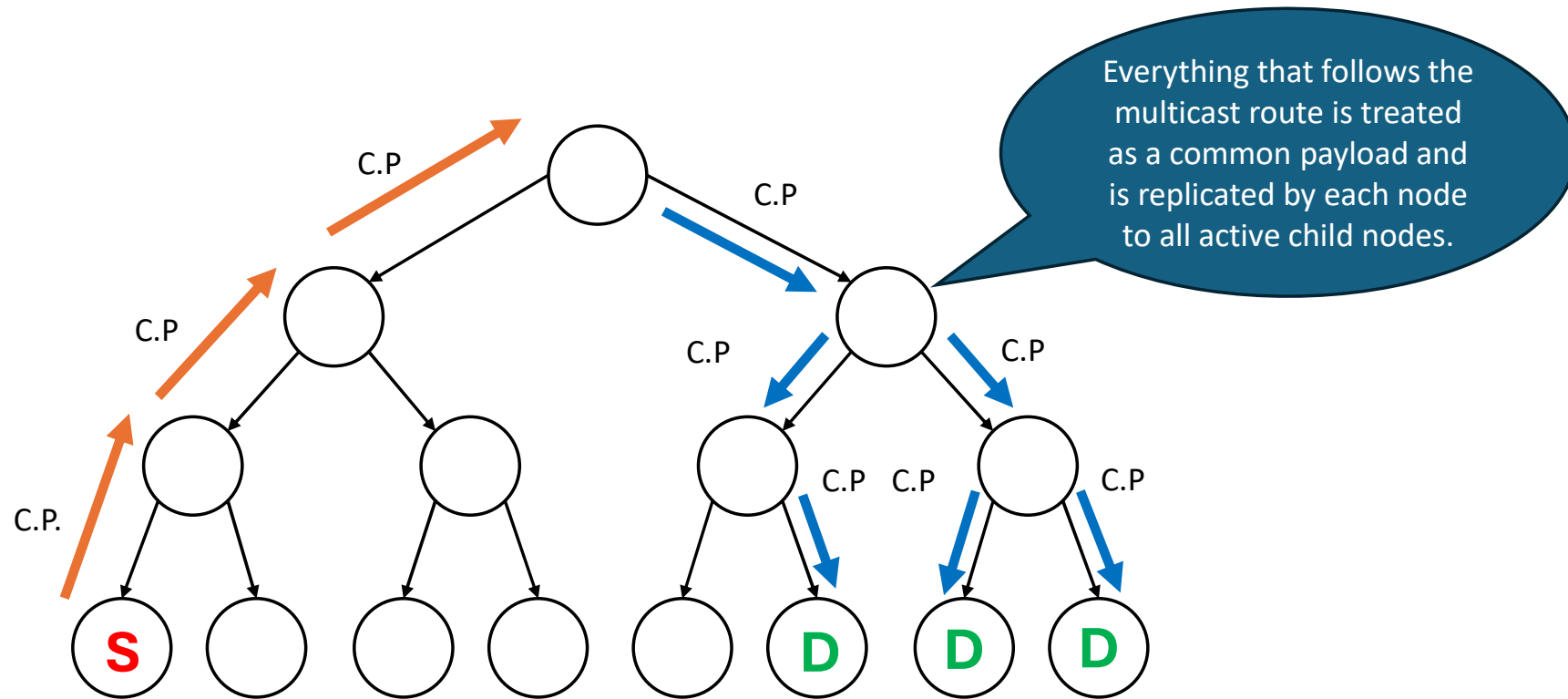
- BW=32 bits
- Must avoid deadlocks
- **Binary tree for simplicity**

Multicast route's flit structure:



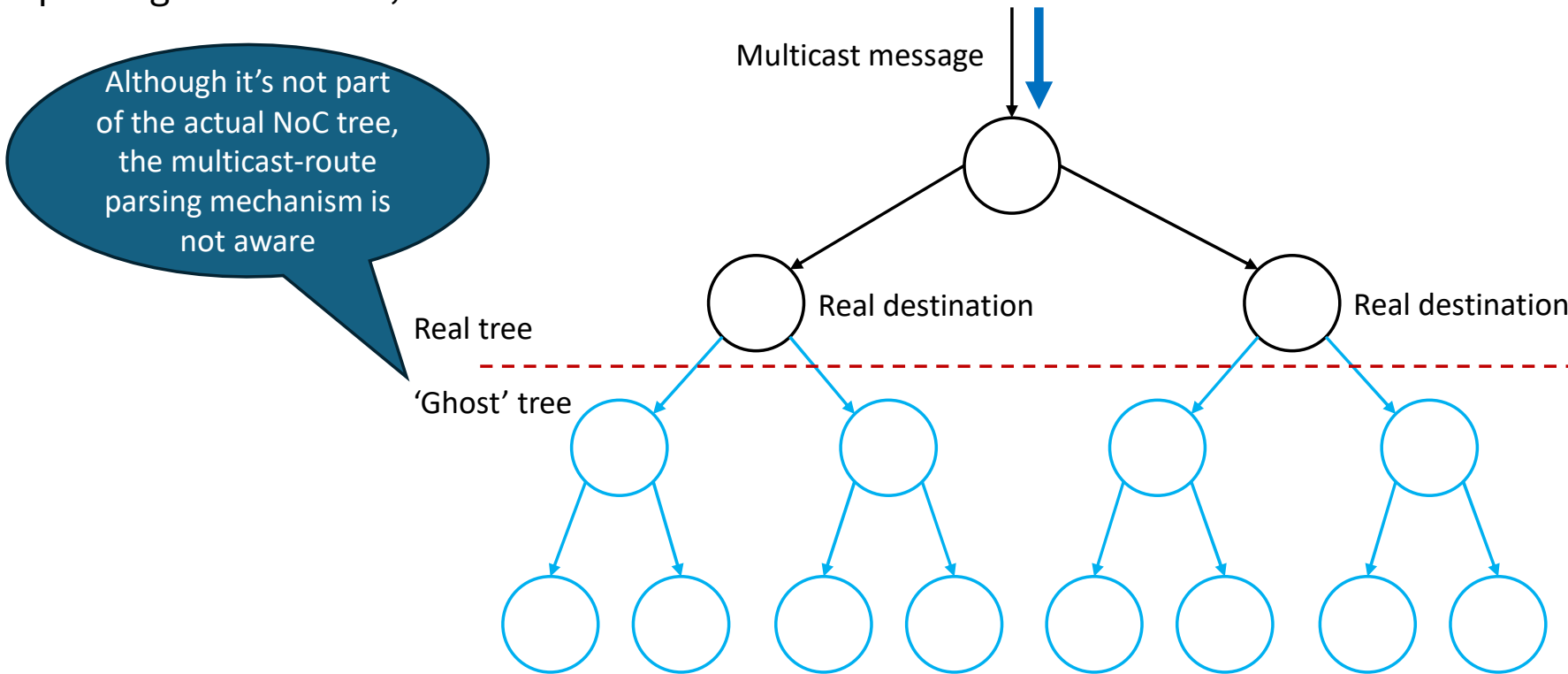
Design of a quad-tree NoC –Multicasting

- BW=32 bits
- Must avoid deadlocks
- **Binary tree for simplicity**



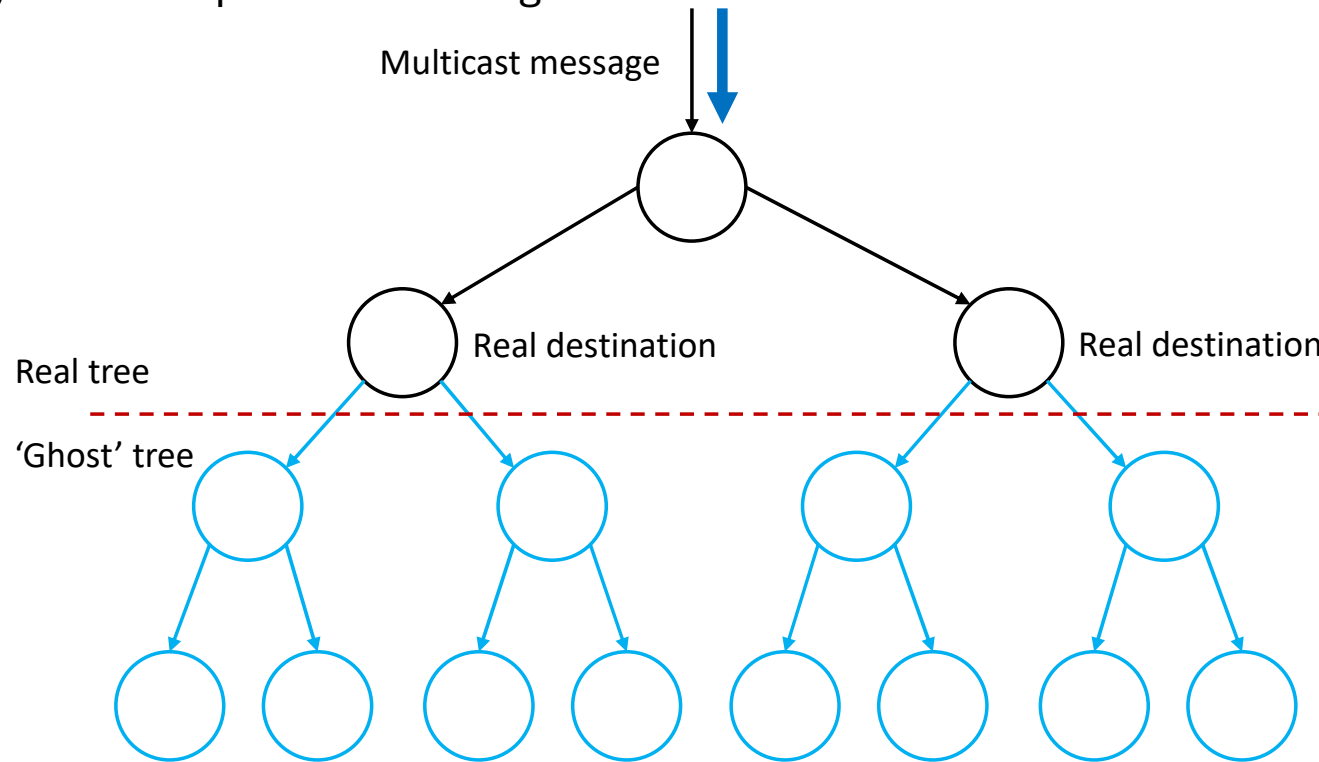
Design of a quad-tree NoC –Multicasting Different Payload

This route encoding scheme allows different payloads to be delivered to different destinations in an elegant manner without any additional overhead. The key idea is to encode the route as if the tree were deeper than its actual depth. As a result, the destination nodes themselves receive a multicast route, which, due to the parsing mechanism, is different for each destination.



Design of a quad-tree NoC –Multicasting Different Payload

In our chip, we use this property when sending a request to compute the distance between a query and vectors stored in memory tiles. The query ID is sent as common payload, whereas the vector addresses are sent to each memory tile (the real destinations) through the destinations of the "ghost" tree, leveraging the fact that the memory address space is itself organized as a tree.



Design of a quad-tree NoC –Multicasting Benefits and Limitations

- **Benefits**

- **Address compression:** when multicasting to all destinations in a 6-level quad-tree (as in our design), only 1.33 bits per destination are required
- **Address scaling:** address's length scales with the number of destinations
- **Personalized payload:** each destination tile can receive a destination-specific payload in addition to a common payload

- **Limitations**

- **Tree encoding:** destination-specific payloads must be encoded according to a tree structure.
- **Breadth-first:** the multicast route is encoded level by level. As a result, the destination tiles cannot reconstruct their complete ghost addresses, and therefore their complete data, until the entire multicast route has been received. However, it is important to note that the portion of the multicast route consumed at each level grows exponentially (with a branching factor of four). As a result, the first useful data can be already reconstructed after roughly 25% of the multicast route has been transmitted.

A microscopic view of a silicon chip, showing intricate circuitry and various colored regions (red, green, blue, yellow) under a light source. A large, solid blue rectangular overlay is positioned in the center of the image, containing the text 'Expanding the ez130 library' in white.

Expanding the ez130 library

Expanding the ez130 library -Overview

- **Context**

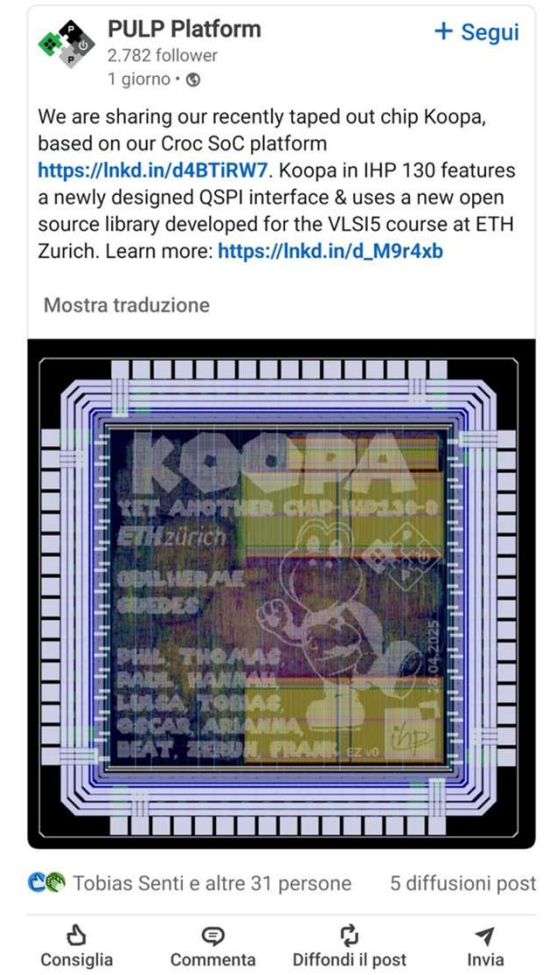
- **VLSI5:** first time the course was offered. Very limited slots.
- **Scope:** expand an existing open source 130nm library of PULP

- **Goal**

- Design 6 new standard cells

- **Workflow**

- **Schematic design:** design exploration, logic restructuring
- **Stick diagram:** share diffusions, optimize connections
- **Physical design:** Cadence
- **Characterization:** PEX, generate .lef and .gds
- **Post-layout verification:** implemented a small design that used them



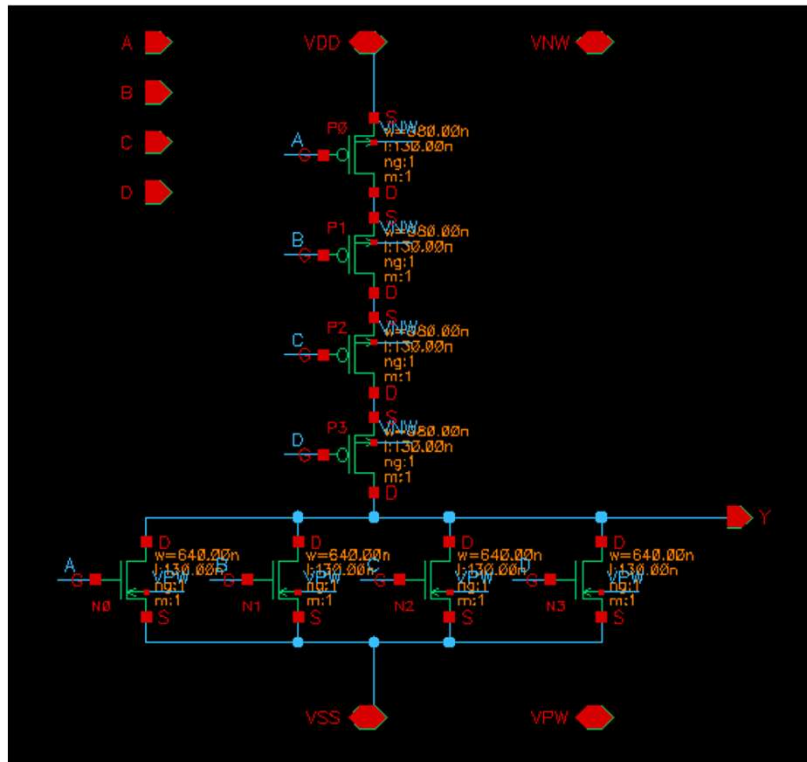
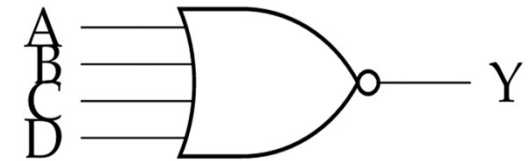
Our cells are already getting used

Expanding the ez130 library -NOR4X2 / X1

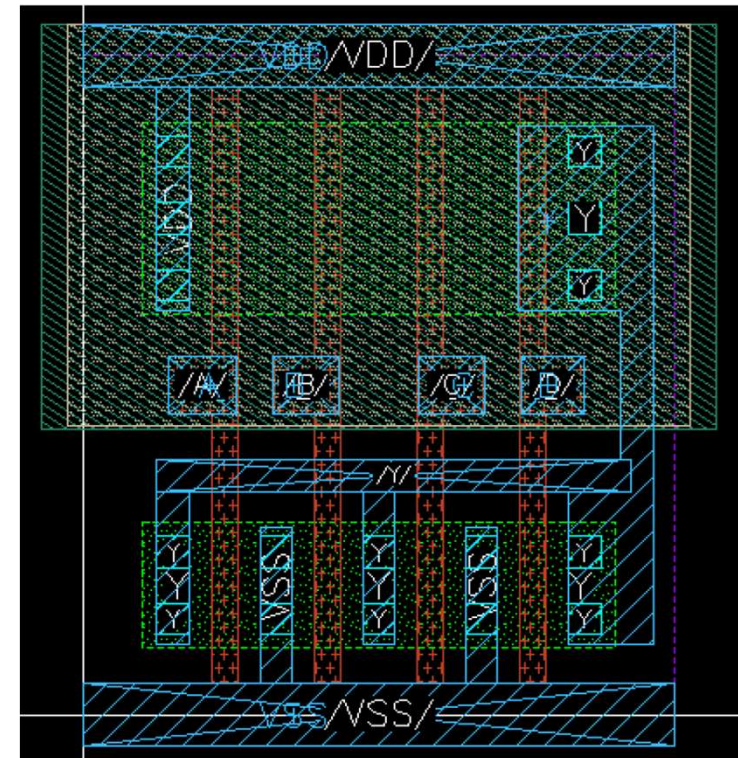
Logic function: $\overline{A + B + C + D}$

N° of transistors: 8

Area: 9.8784 μm^2



NOR4X2 schematic



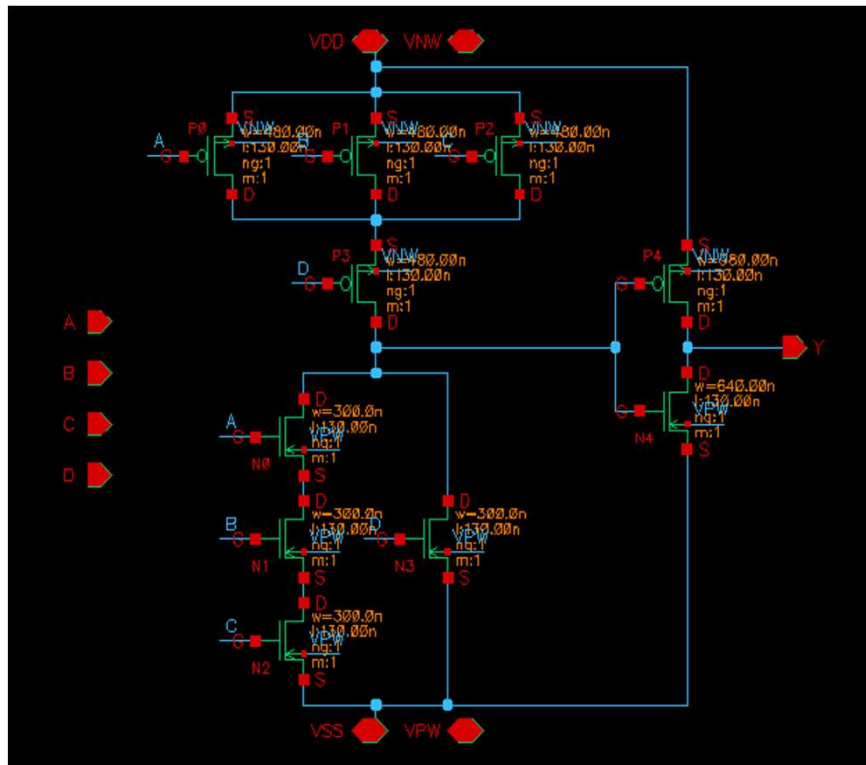
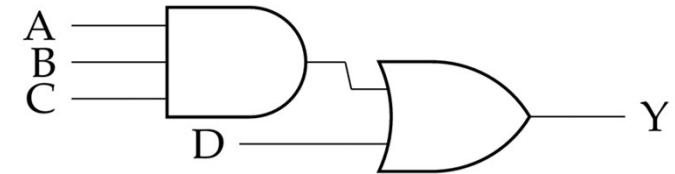
NOR4X2 layout

Expanding the ez130 library –AO31X2 / X1

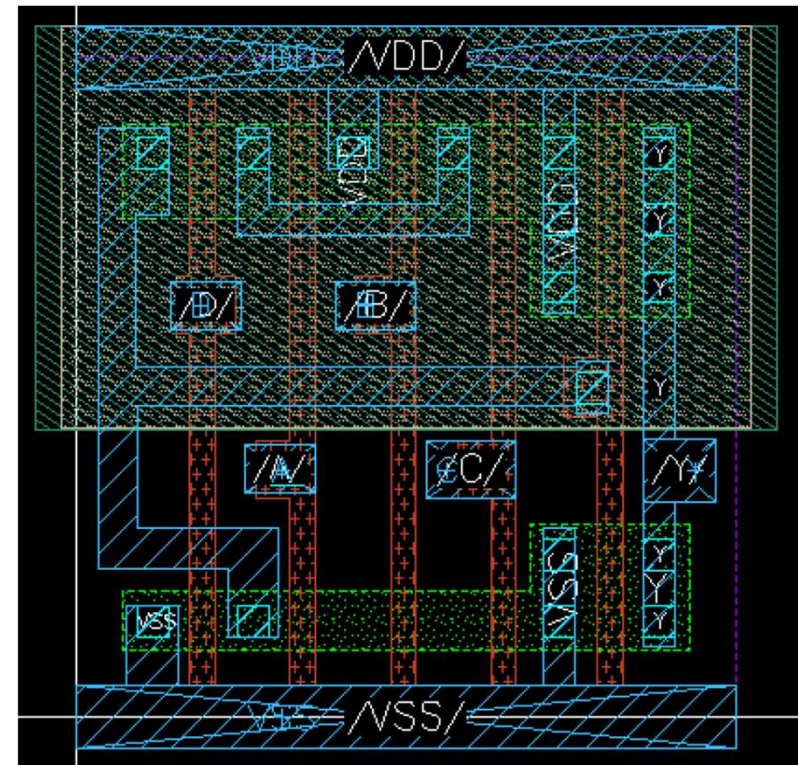
Logic function: $A \cdot B \cdot C + D$

N° of transistors: 10

Area: 11.2896 μm^2



AO31X2 schematic



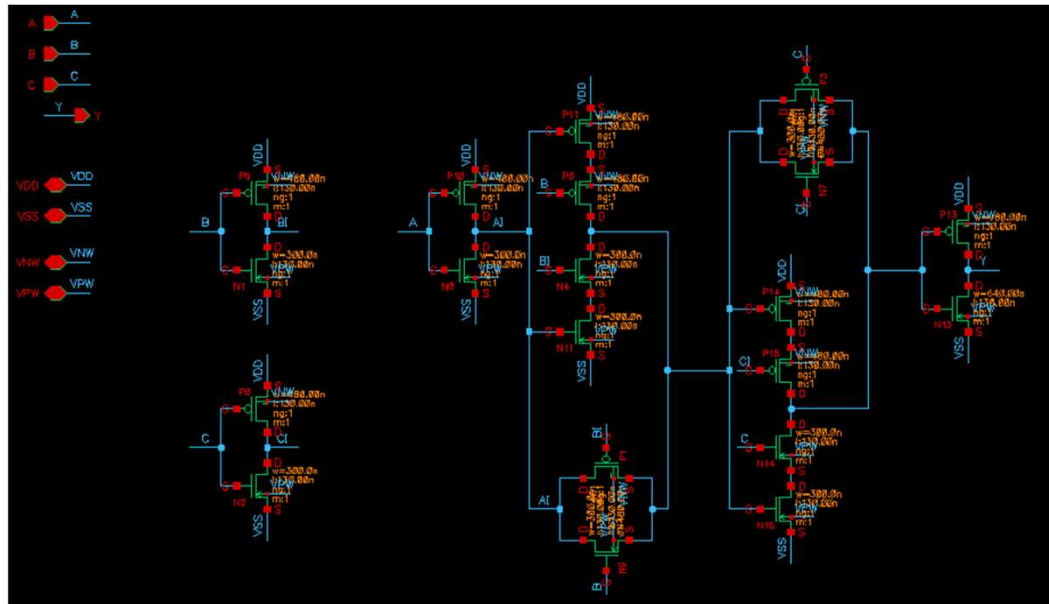
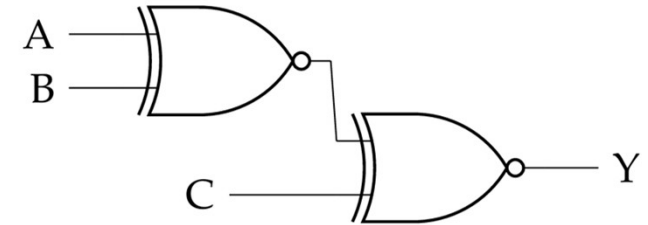
AO31X2 layout

Expanding the ez130 library –XNOR3X2 / X1

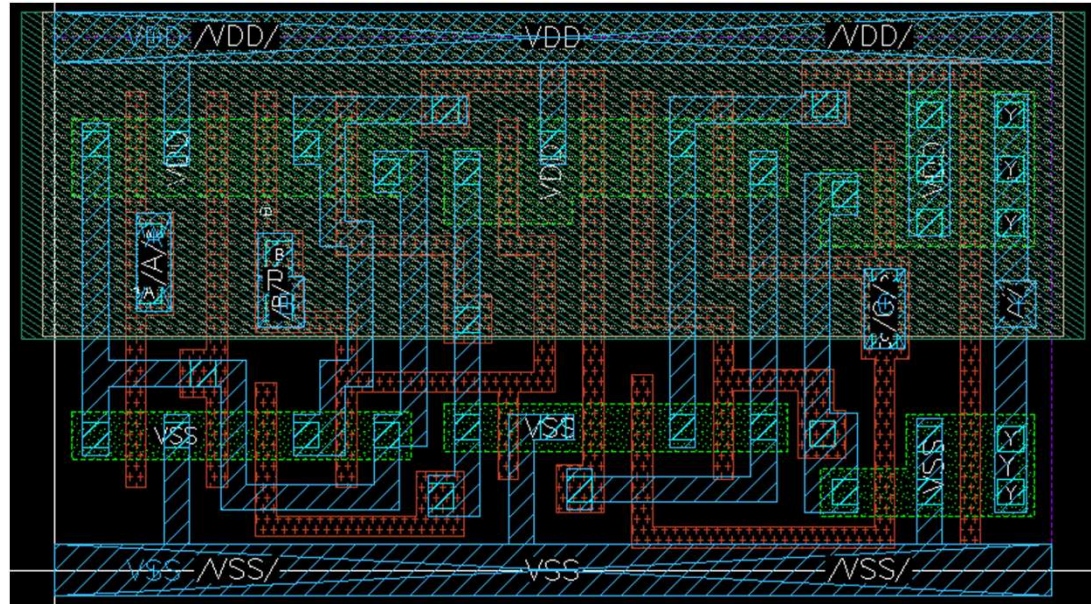
Logic function: $\overline{A \oplus B \oplus C}$

N° of transistors: 20

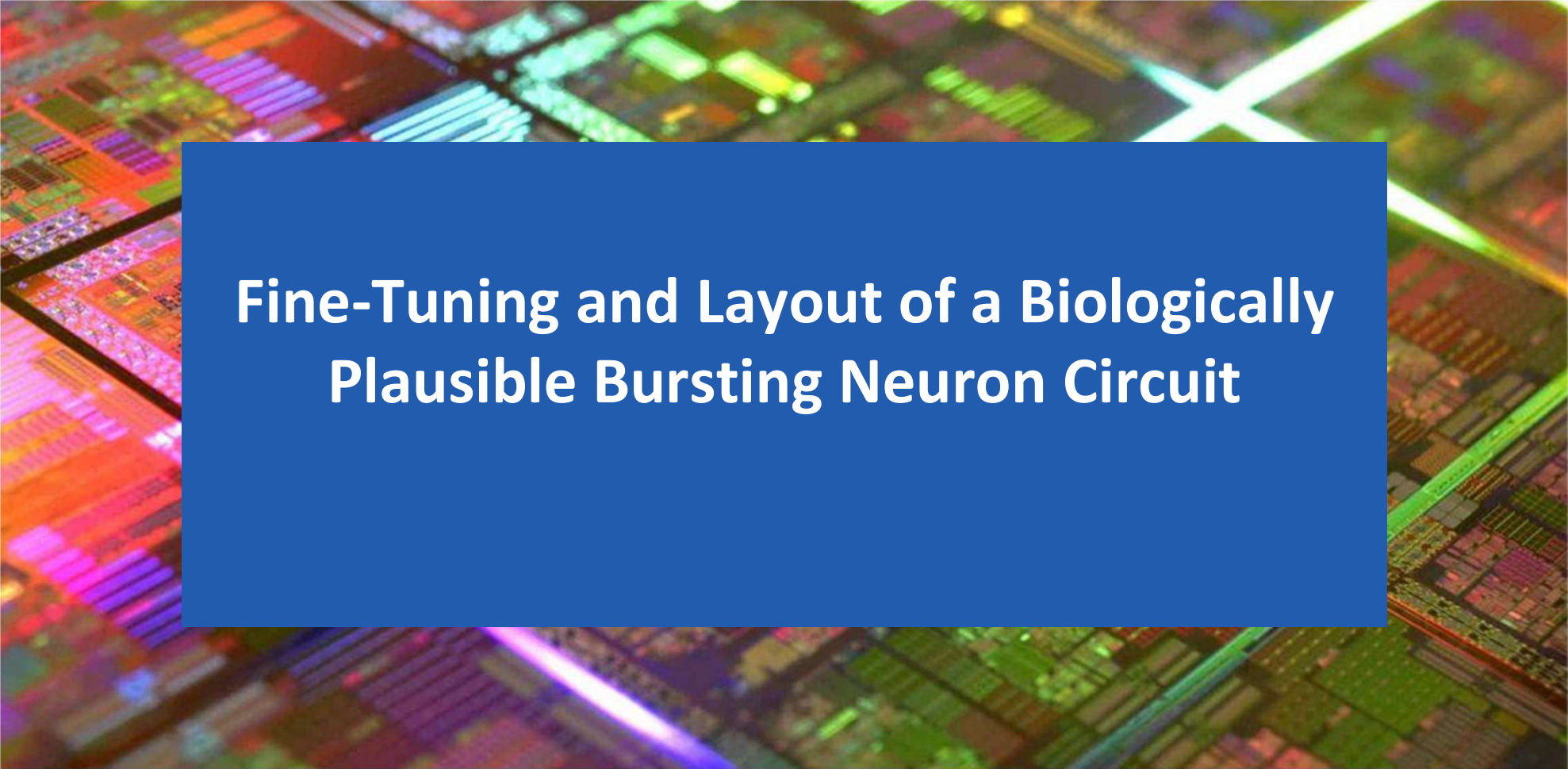
Area: 21.168 μm^2



XNOR3X2 schematic



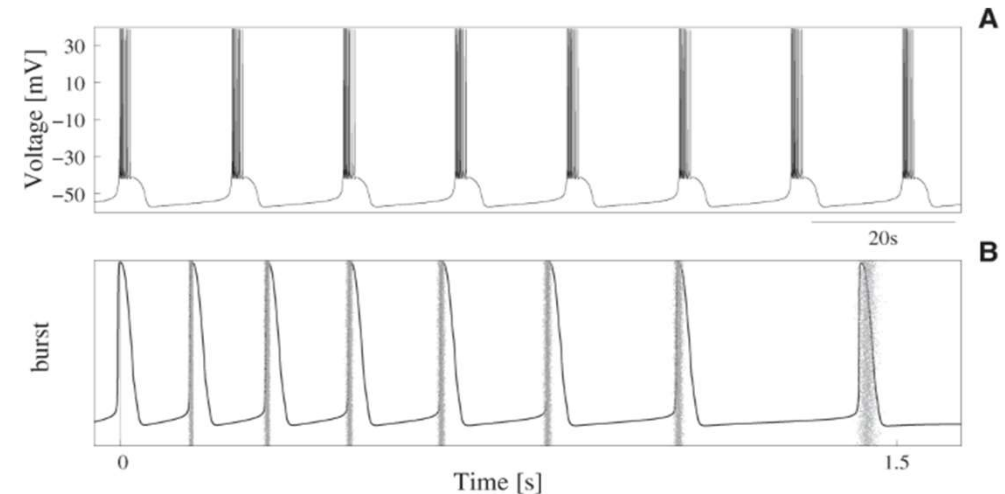
XNOR3X2 layout



Fine-Tuning and Layout of a Biologically Plausible Bursting Neuron Circuit

Bursting Neuron -Overview

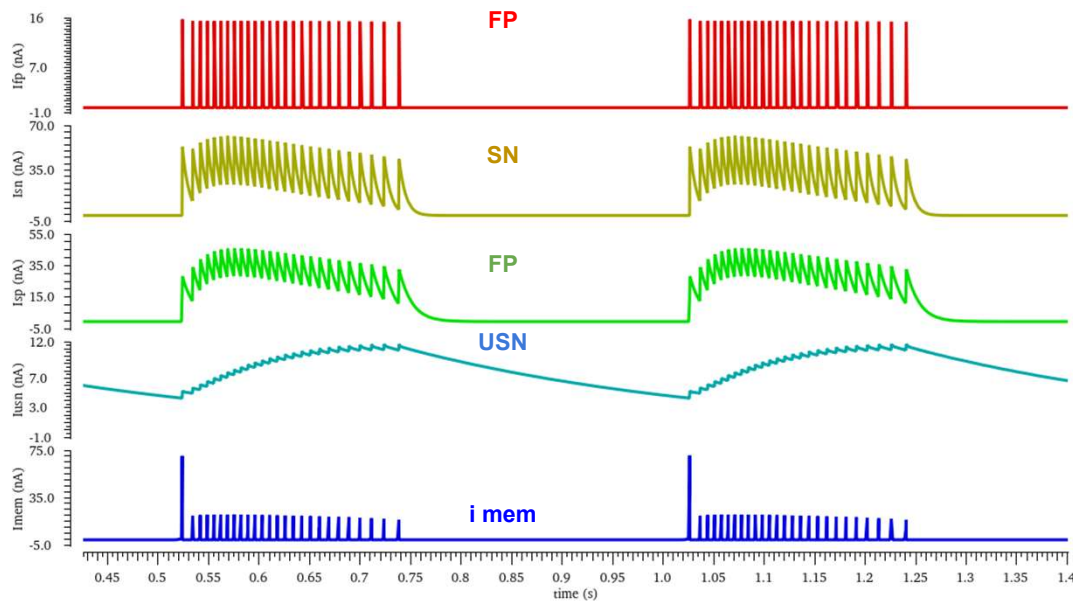
- **Context**
 - **Neuromorphic II:** analog IC design
 - **Scope:** learn of to design the layout of analog circuit
- **Goal**
 - Obtain a functioning layout of the given circuit
- **Workflow**
 - **Tuning:** find the right parameters and transistor sizes
 - **Simulations:** make sure the spiking activity is happening
 - **Physical implementation:** layout of the circuit
 - **Post-layout simulations:** Montecarlo simulations



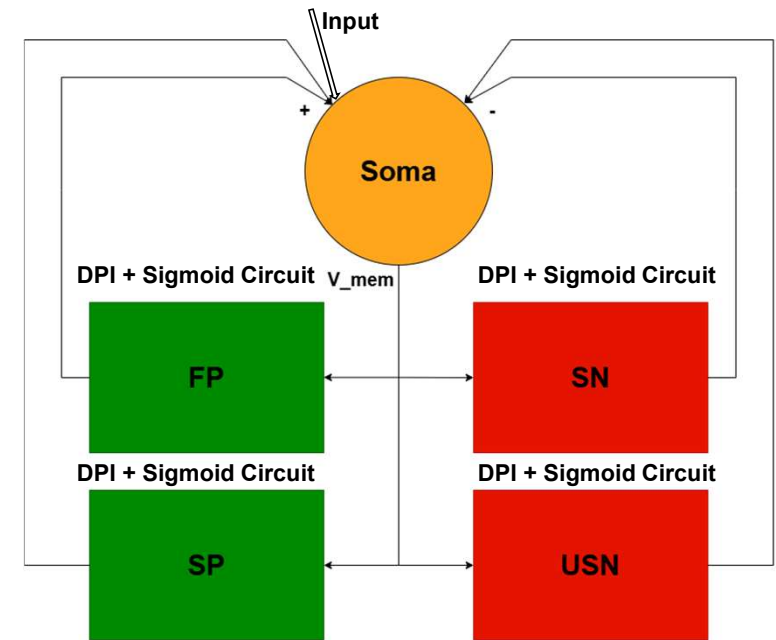
Bursting activity in biological neurons

Bursting Neuron –Circuit overview

- Input: DC current
- Output: bursts in membrane current
- 4 Feedback mechanisms (DPI + Sigmoid):
 - FP (fast positive), SN (slow negative), SP (slow positive), USN (ultra slow negative)

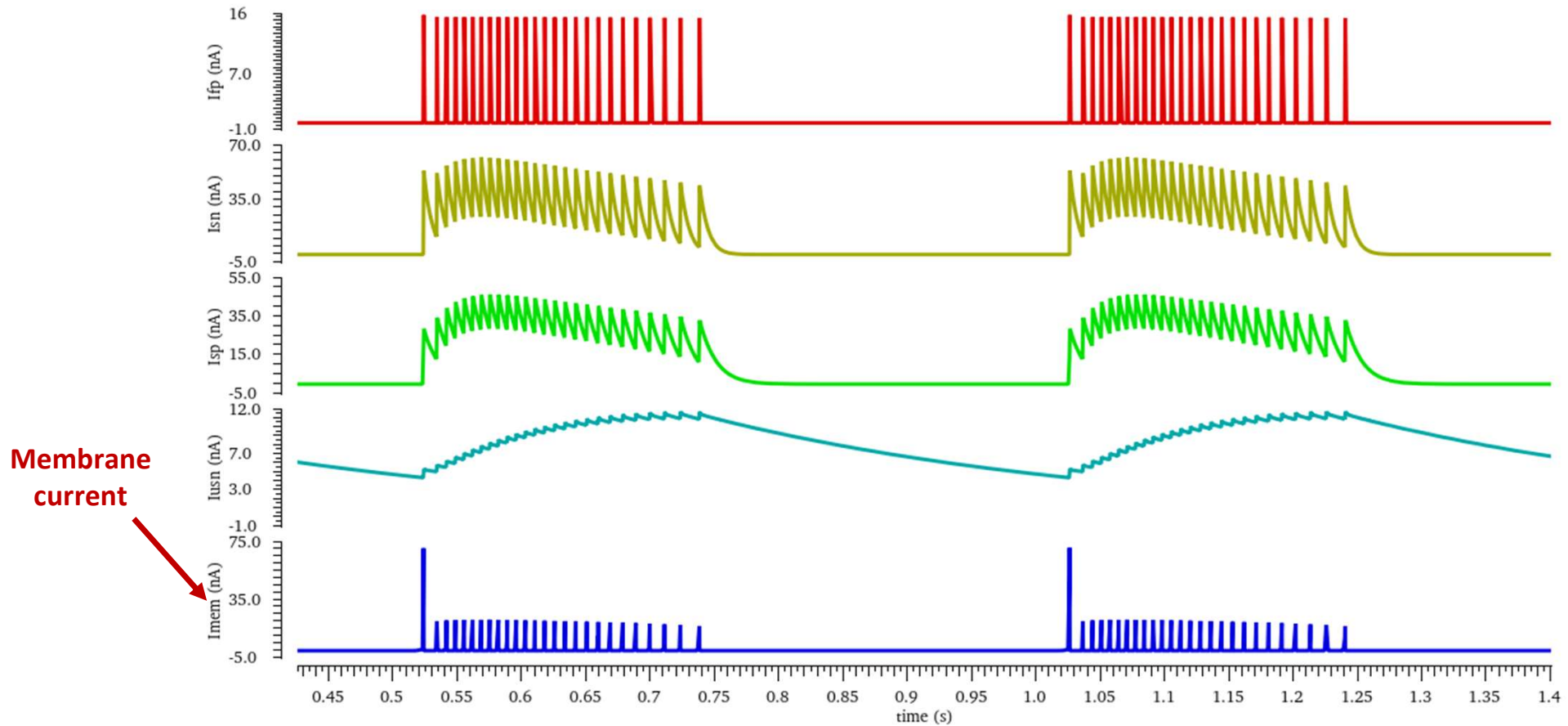


Bursting activity in our circuit



Model used

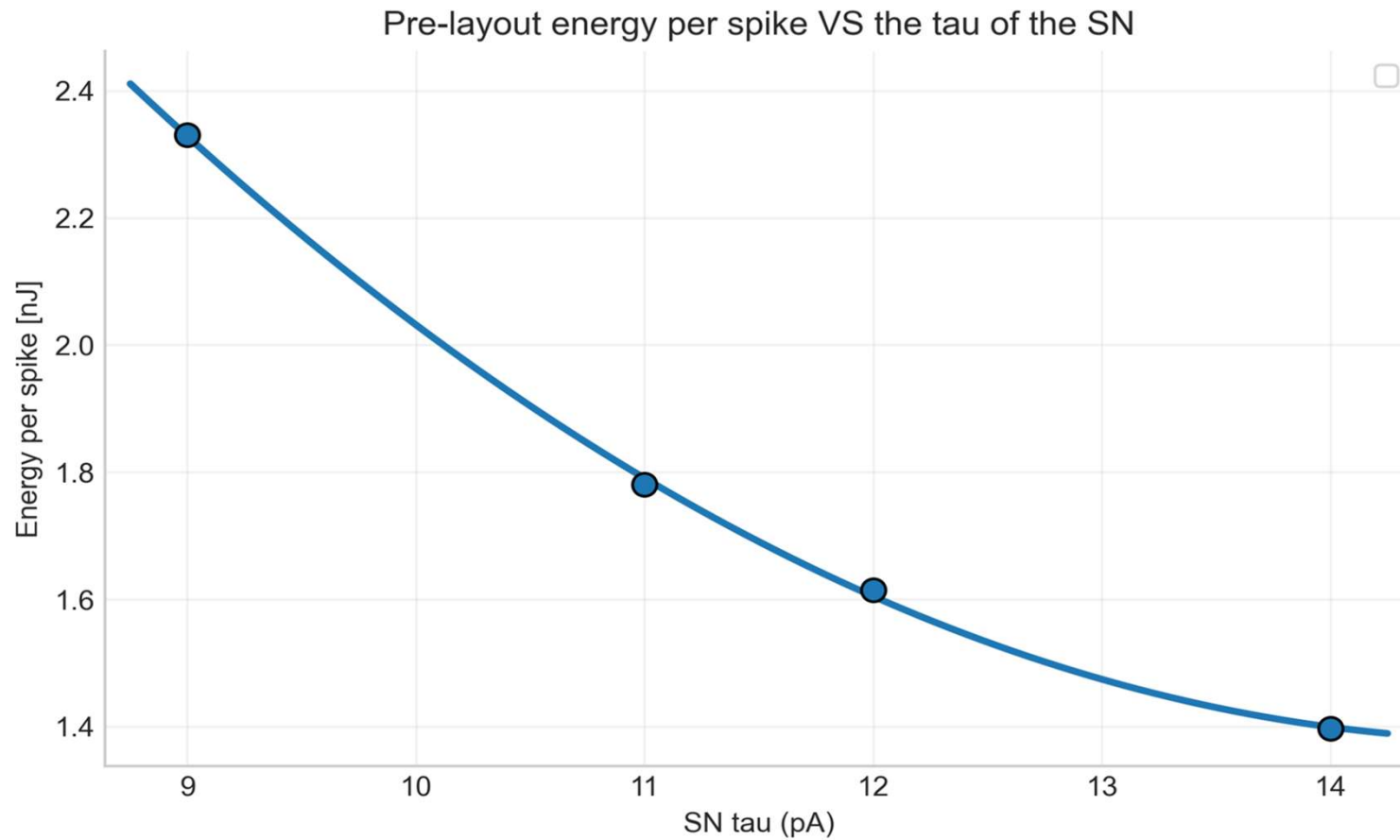
Bursting Neuron –Bursting activity



Bursting activity in our circuit

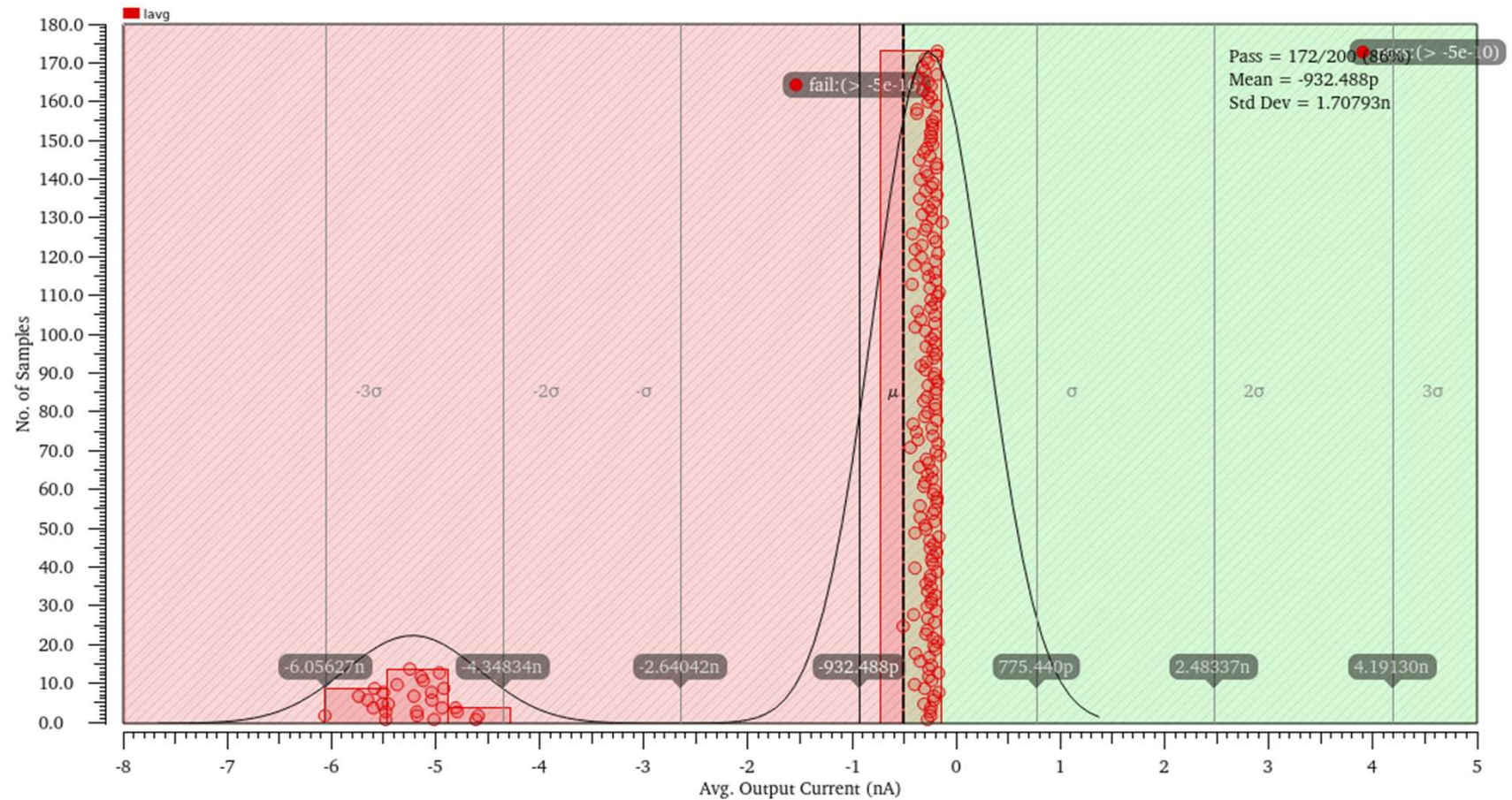
Bursting Neuron –Power analysis Pre-layout

The energy consumption for each spike decreases with the tau of the SN since it makes the spikes more narrow.



Bursting Neuron –Montecarlo Pre-layout

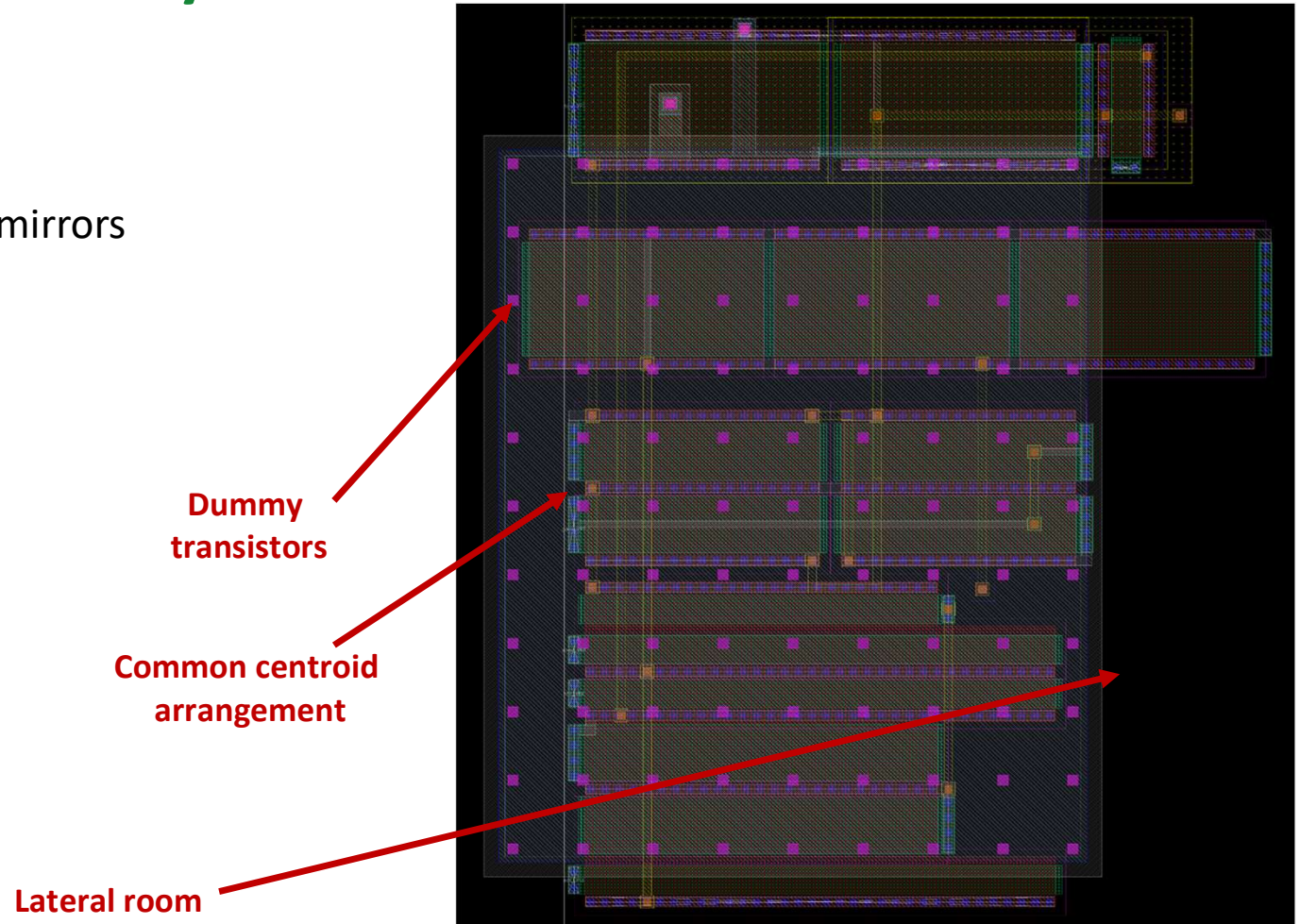
Passing samples: 86%



Montecarlo simulation results for mismatch + process variations

Bursting Neuron –Feedback layout

- 4 copies of the feedback block
- Features:
 - Centroid arrangement for current mirrors
 - Stripe of dummy transistor
 - Lateral room for the Soma's block



Layout of one of the feedback mechanisms

Bursting Neuron –Final layout

- **Structure**

- 4 feedback blocks
- Soma in the center
- Two rows of PMOS
- Central block of NMOS

- **Guard rings:**

- P guard rings around NMOS
- N guard rings around PMOS

- **Dummy transistors**

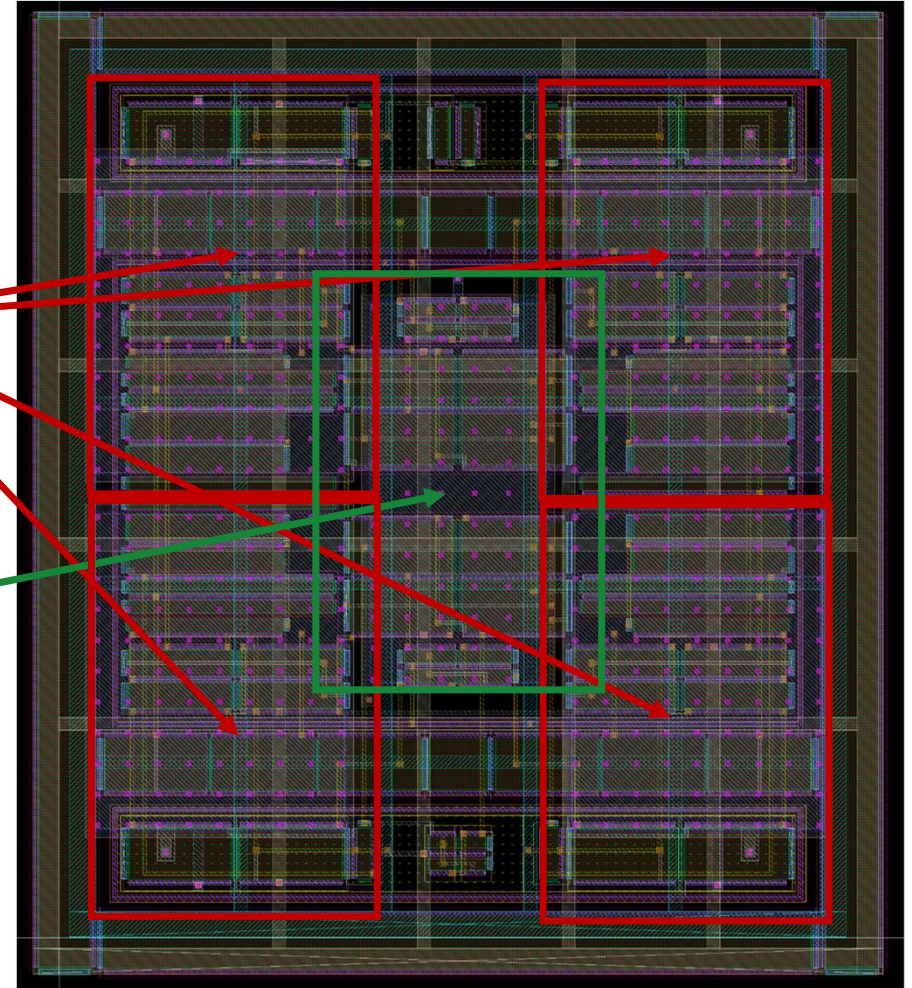
- To separate PMOS and NMOS
- All around

- **Power grid**

- **Area: $75.66 \times 67.02 \mu\text{m}^2$**

4 Feedbacks

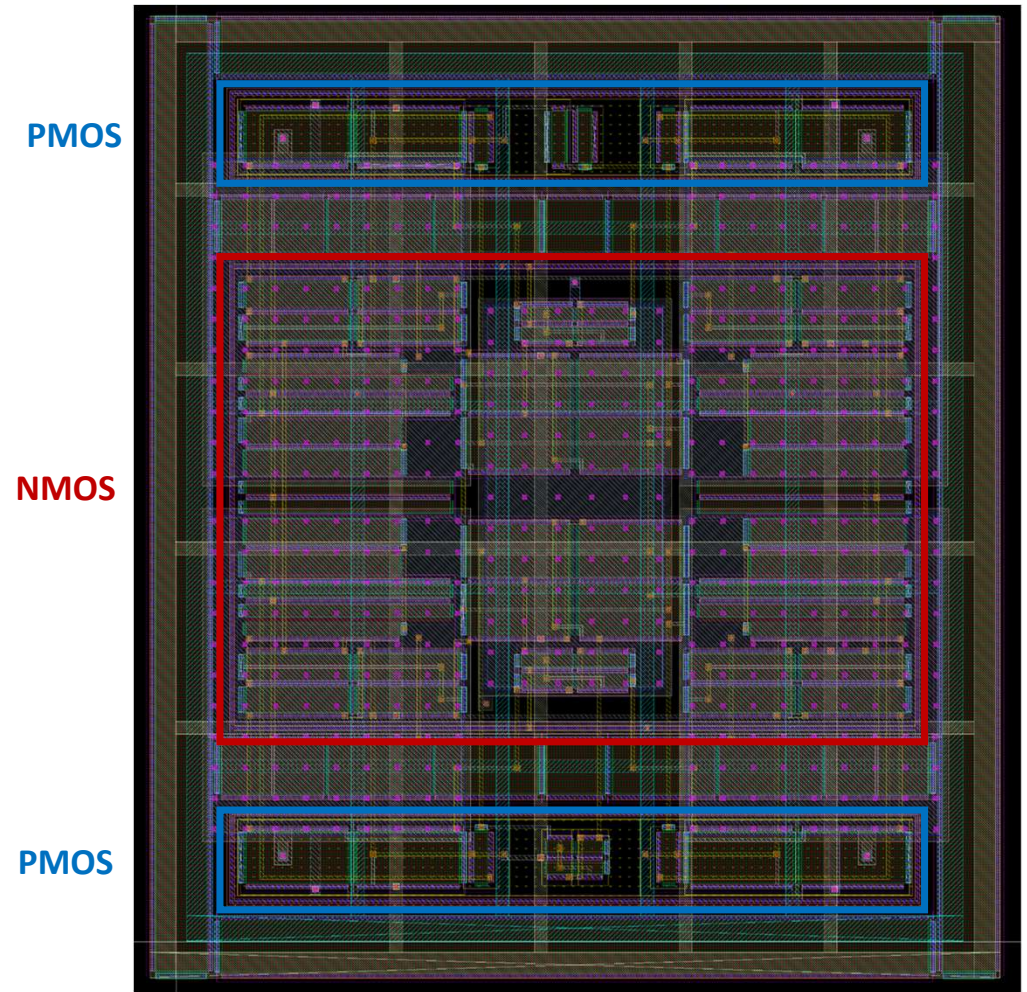
Soma



Layout of the final neuron

Bursting Neuron –Final layout

- **Structure**
 - 4 feedback blocks
 - Soma in the center
 - Two rows of PMOS
 - Central block of NMOS
- **Guard rings:**
 - P guard rings around NMOS
 - N guard rings around PMOS
- **Dummy transistors**
 - To separate PMOS and NMOS
 - All around
- **Power grid**
- **Area: $75.66 \times 67.02 \mu\text{m}^2$**

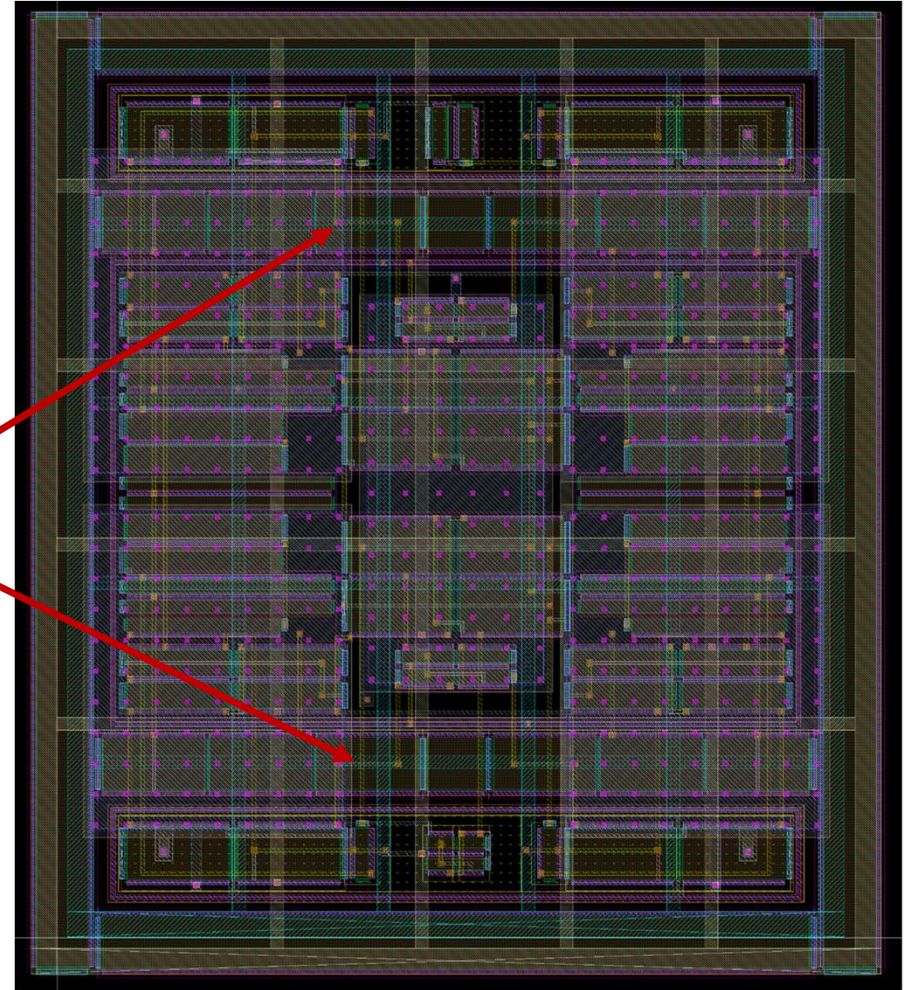


Layout of the final neuron

Bursting Neuron –Final layout

- **Structure**
 - 4 feedback blocks
 - Soma in the center
 - Two rows of PMOS
 - Central block of NMOS
- **Guard rings:**
 - P guard rings around NMOS
 - N guard rings around PMOS
- **Dummy transistors**
 - To separate PMOS and NMOS
 - All around
- **Power grid**
- **Area: $75.66 \times 67.02 \mu\text{m}^2$**

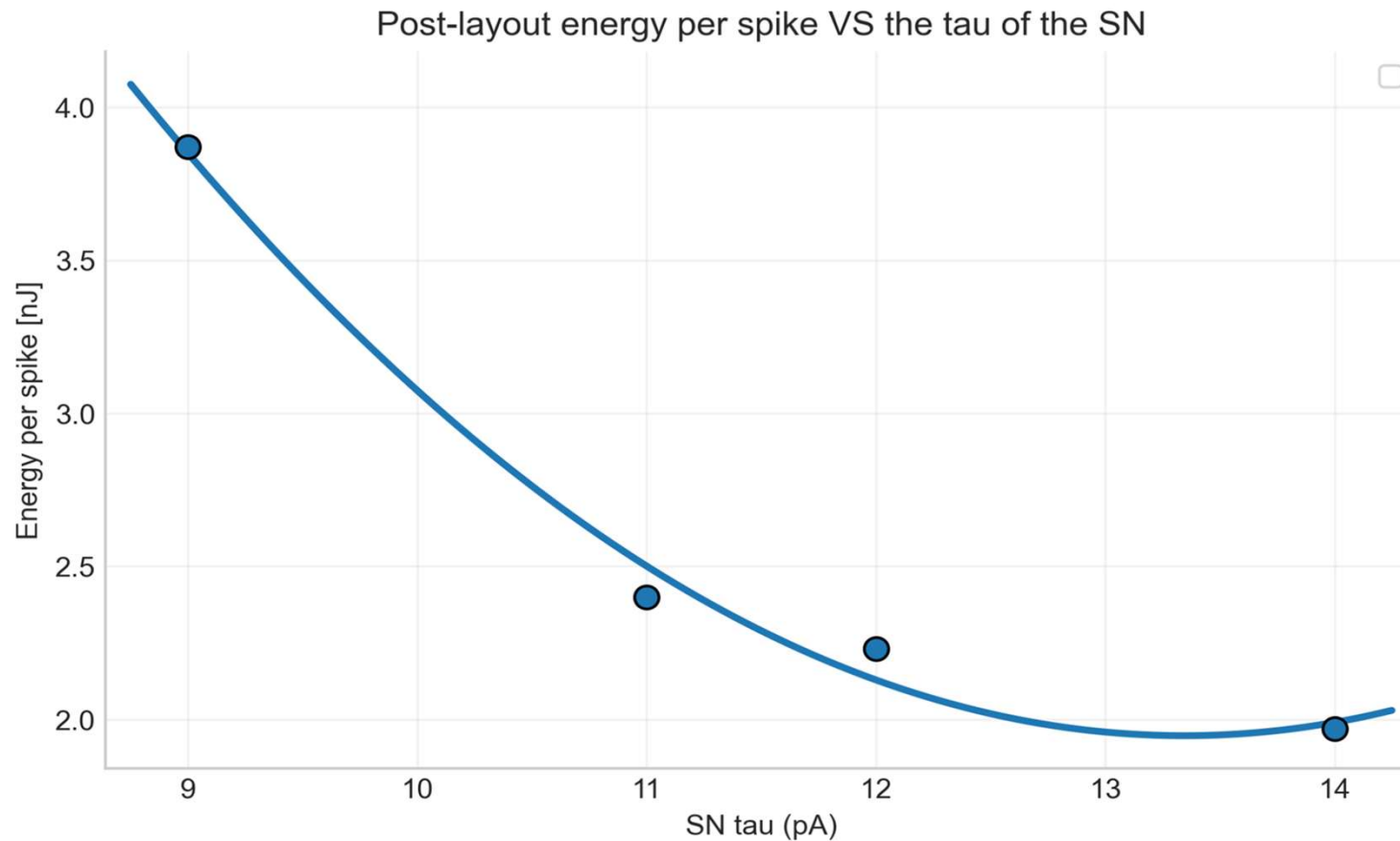
Dummy transistors



Layout of the final neuron

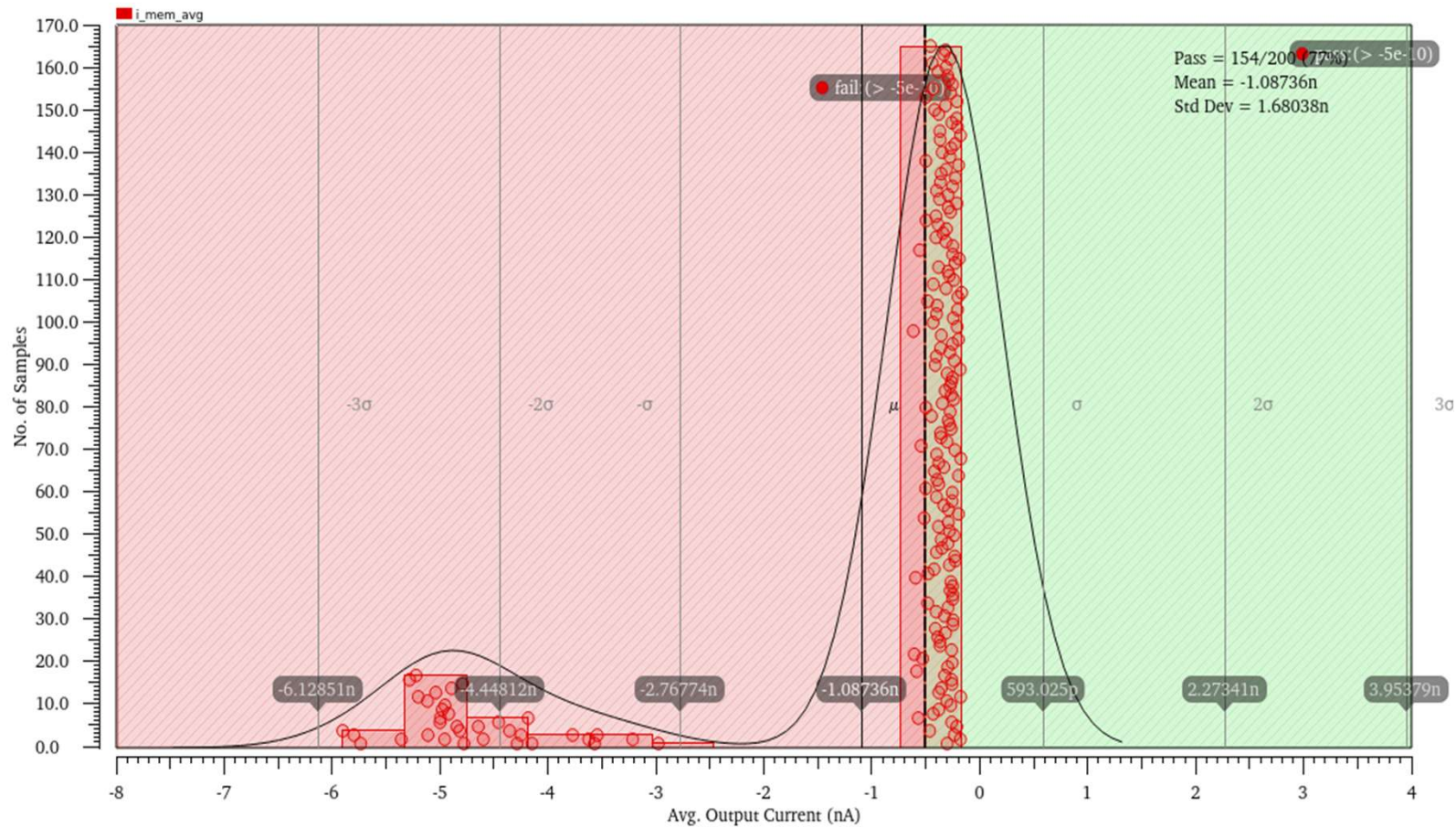
Bursting Neuron –Power analysis Post-layout

As expected, the power consumption is higher post layout, due to the fact that now parasitics are taken into account as well.



Bursting Neuron –Montecarlo Post-layout

Passing samples: 77%



A close-up, high-magnification photograph of a microchip. The chip's surface is covered in a complex grid of tiny, rectangular components, likely transistors or memory cells. The colors are vibrant and varied, including shades of red, orange, yellow, green, and blue, which are likely due to the different materials and processes used in the chip's fabrication. The lighting is bright, creating a high-contrast image that highlights the intricate details of the chip's architecture.

Designing a SpMM accelerator

SpMM Accelerator-Overview

- **Context**

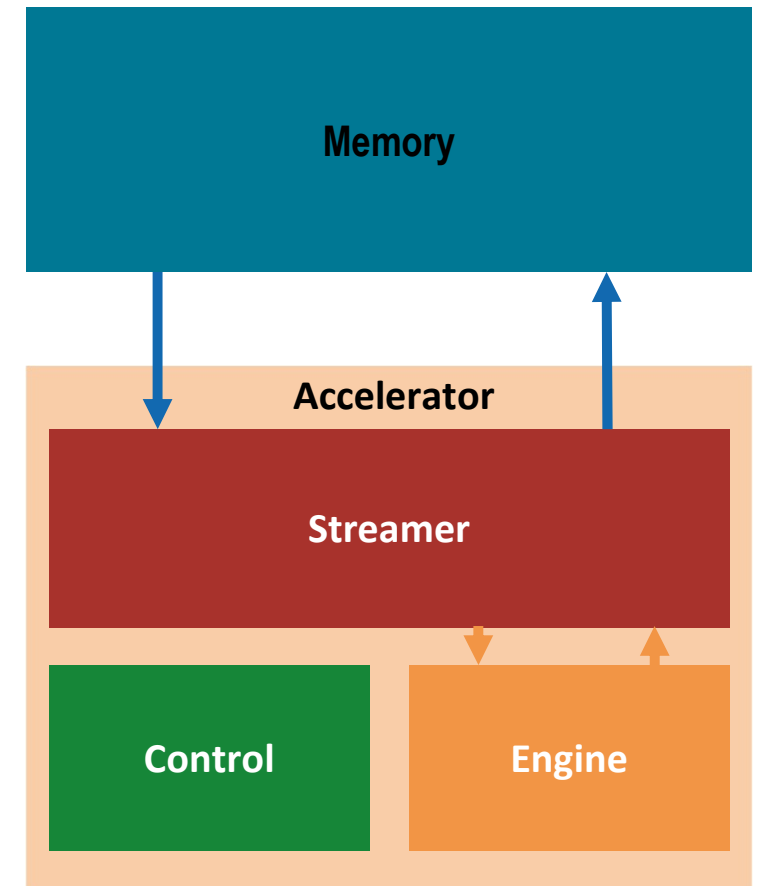
- **Semester thesis:** under Institute for Integrated Systems (IIS)
- **Scope:** learn the higher-end of an ASIC workflow

- **Goal**

- Have the functioning RTL of the sparsity-aware streamer of the accelerator

- **Workflow**

- **Analytical evaluation:** Gustavson and Outer product dataflows
- **Architecture design:** design the architecture of the streamer
- **Functional verification:** different matrix sizes and sparsity
- **Synthesis:** area results



Structure of the accelerator

Case study

- **Assumptions on data**

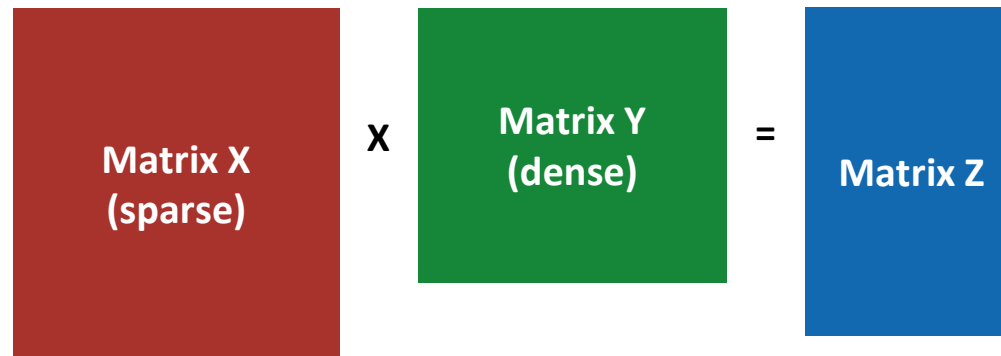
- X sparse
- Y dense

- **X encoded with bitmap format**

- The matrix is stored as its dense elements + a one-hot encoded mask of metadata

- **The engine takes chunks of the matrices**

- The size of the engine determines the size of the tiles of X and Y.



Naming conventions used

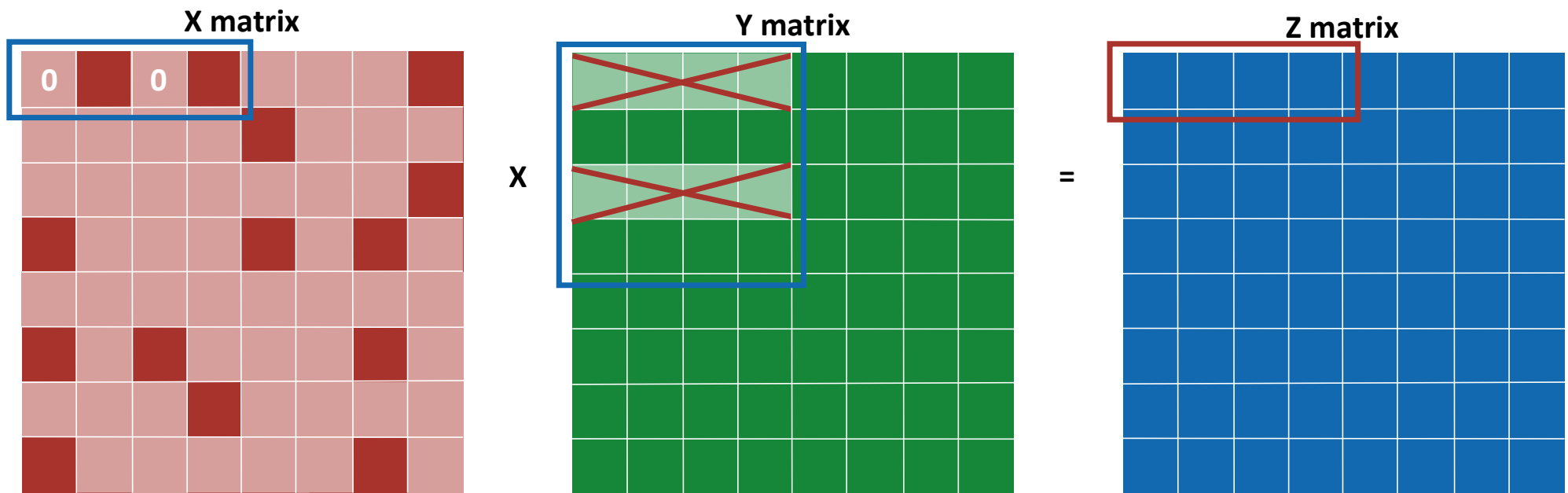
My contributions

- **Analytical evaluation**
 - Compared Gustavson vs. Outer Product dataflows under various reuse strategies for Sparse Matrix Multiplication (SpMM)
- **Sparsity-aware extension of the Hardware Processing Engine's (HWPE) streamer**
 - Integrated sparsity handling logic into the streamer
 - Functionally verified on arbitrary matrix, varying sizes and sparsity levels
- **Design Evaluation**
 - Performance
 - Area overhead

The idea

- **Avoid multiplications by zero**

- If the element in the row of X is a zero, I can skip the correspondent row of Y
- This way I avoid unnecessary loads and multiplications that consume bandwidth and cycles

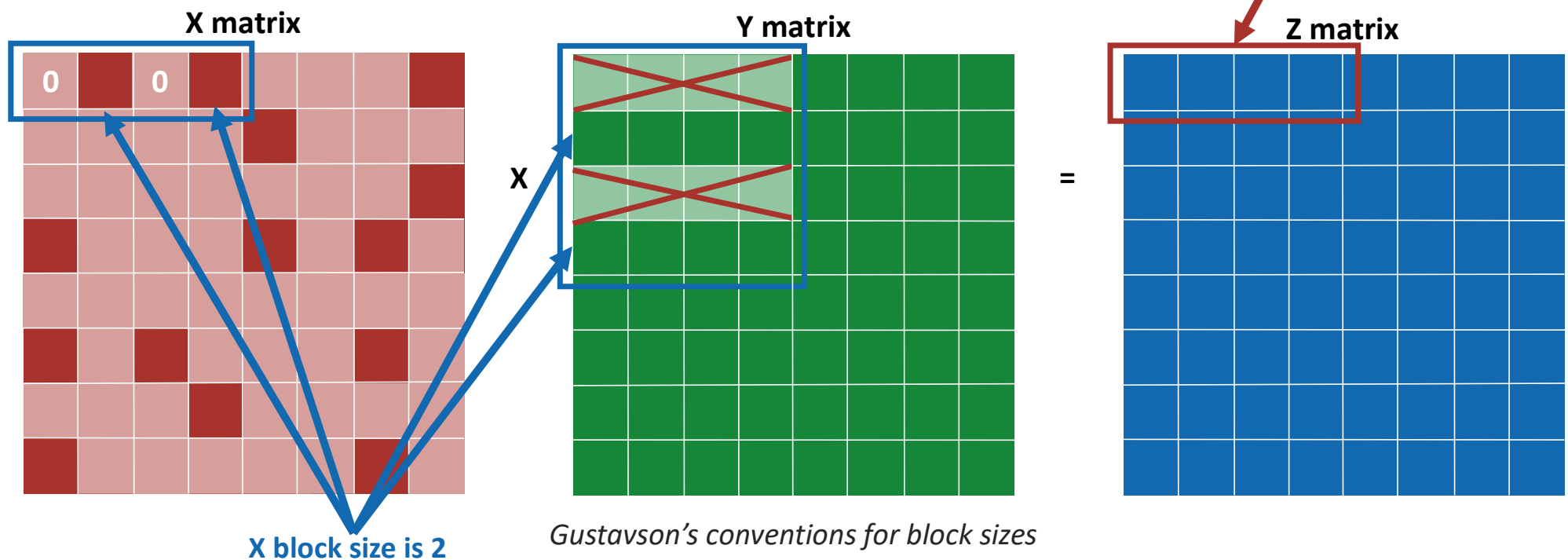


Example with Gustavson

Dataflows options -Gustavson

- **There are 3 possible options**

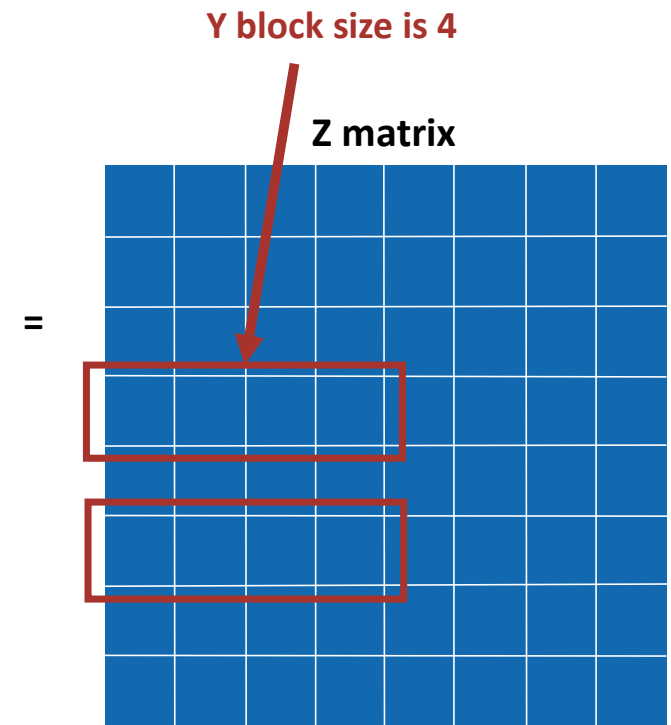
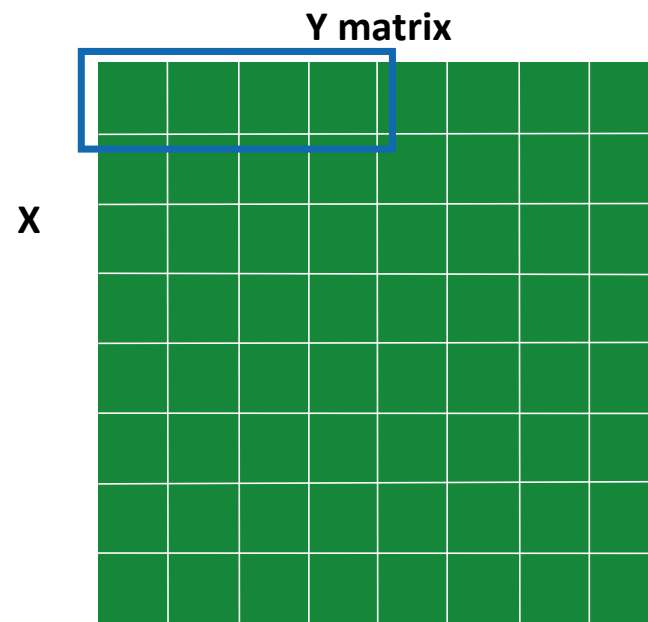
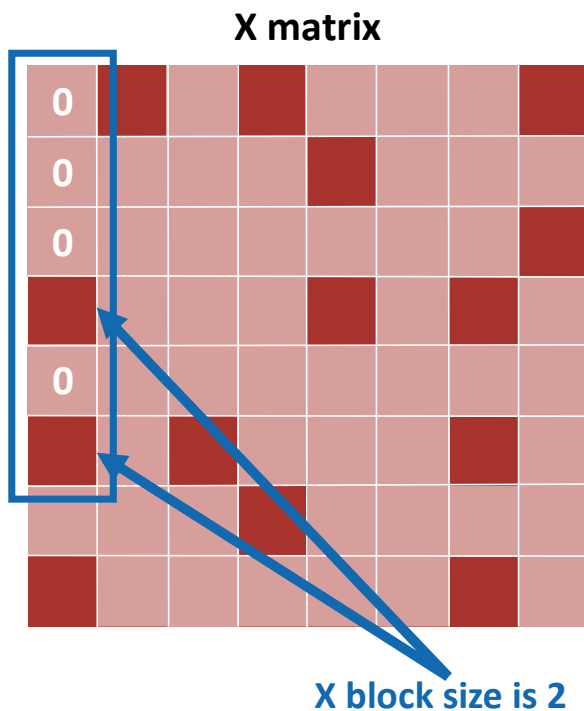
- X reuse: the X block is reused in the inner cycle
- Y reuse: the Y block is reused in the inner cycle
- Z reuse: the Z block is fully computed in the inner cycle



Dataflows options –Outer product

- **There are 3 possible options**

- X reuse: the X block is reused in the inner cycle
- Y reuse: the Y block is reused in the inner cycle
- Z reuse: the Z block is fully computed in the inner cycle

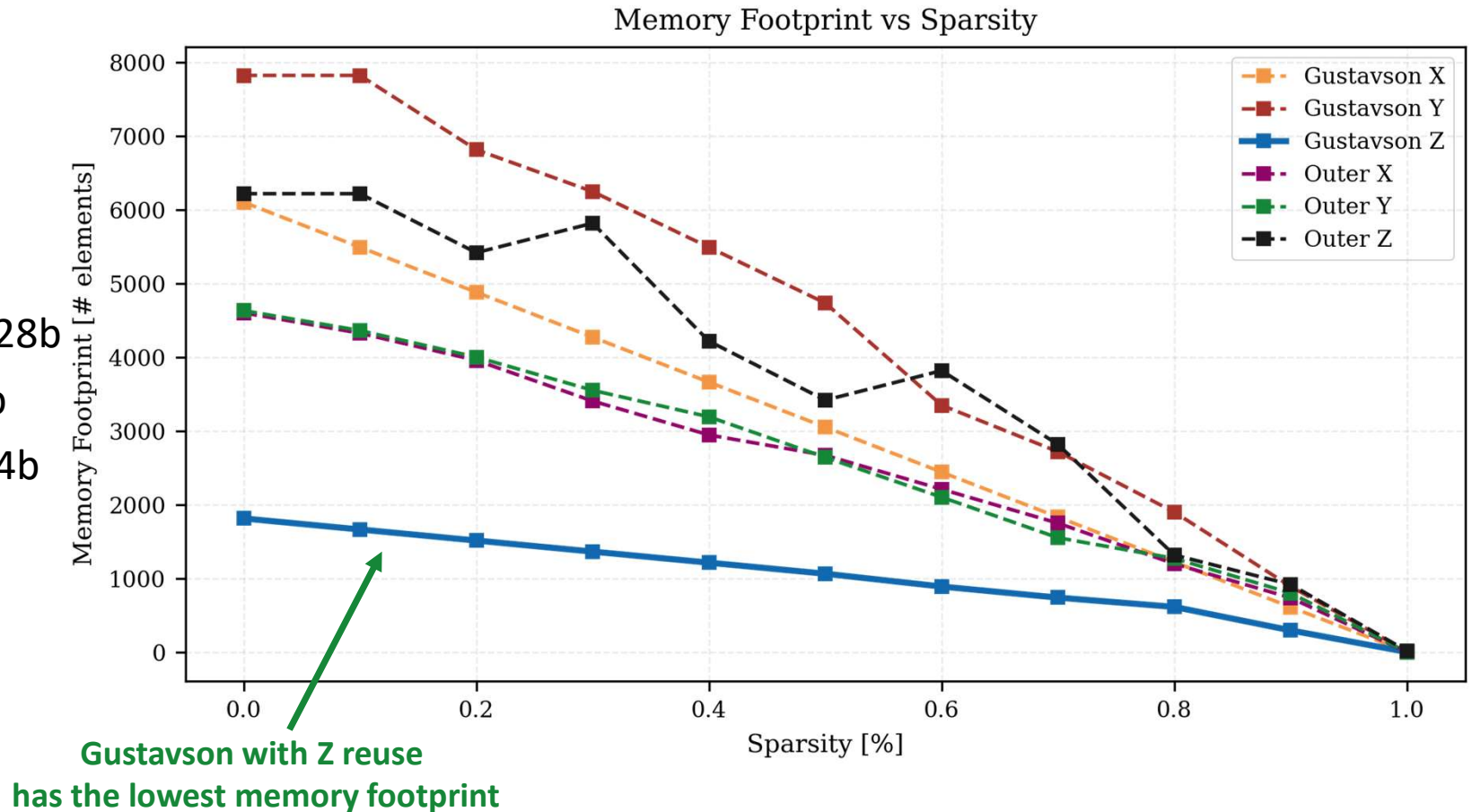


Outer product's conventions for block sizes

Analytical models comparison –Memory footprint

- **Assumptions:**

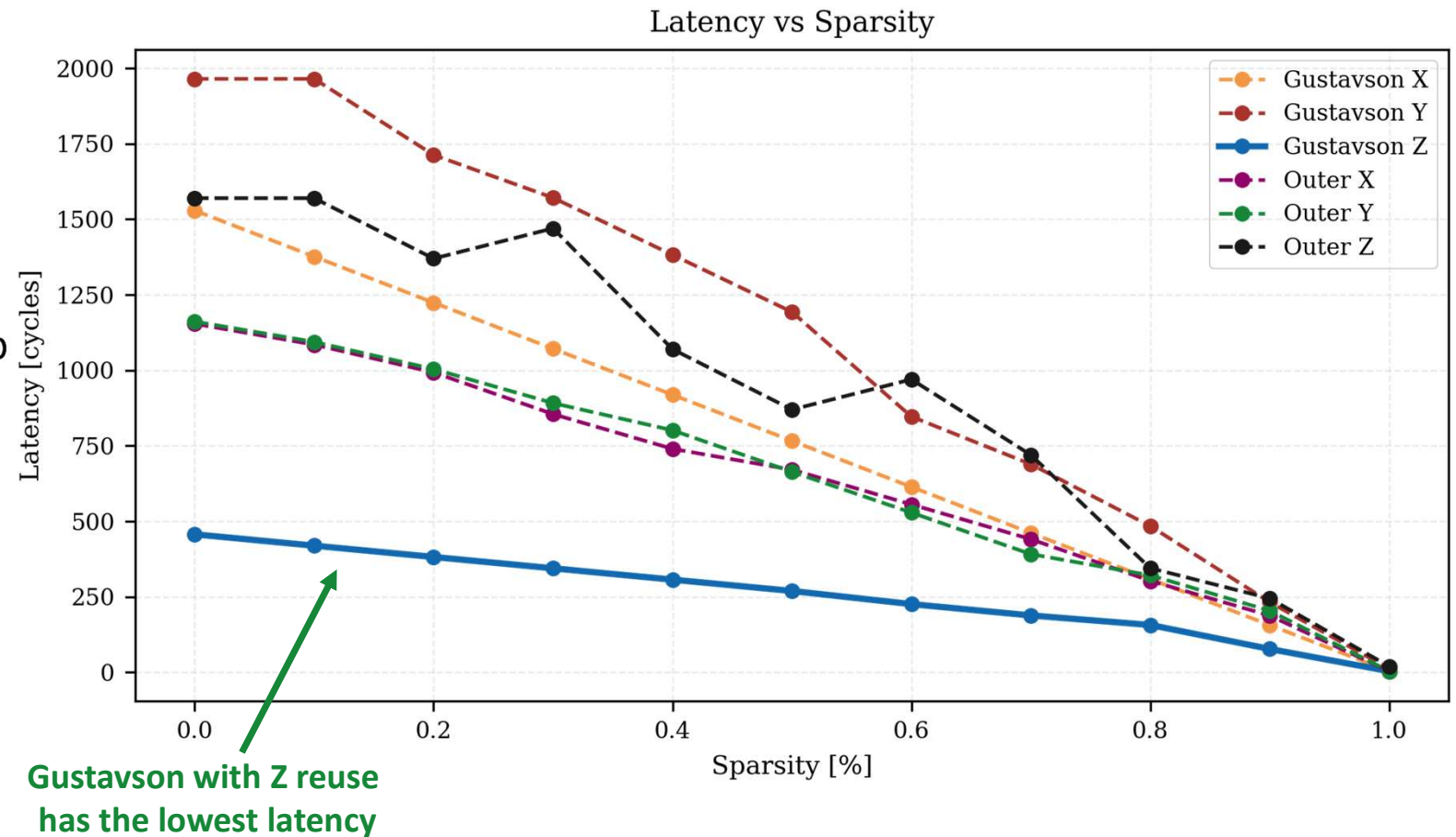
- MAC units = 4
- X rows = 32
- Y columns = 32
- Inner dim. = 32
- Metadata chunk = 128b
- Input data size = 32b
- Output data size = 64b
- BW = 128b



Analytical models comparison –Latency

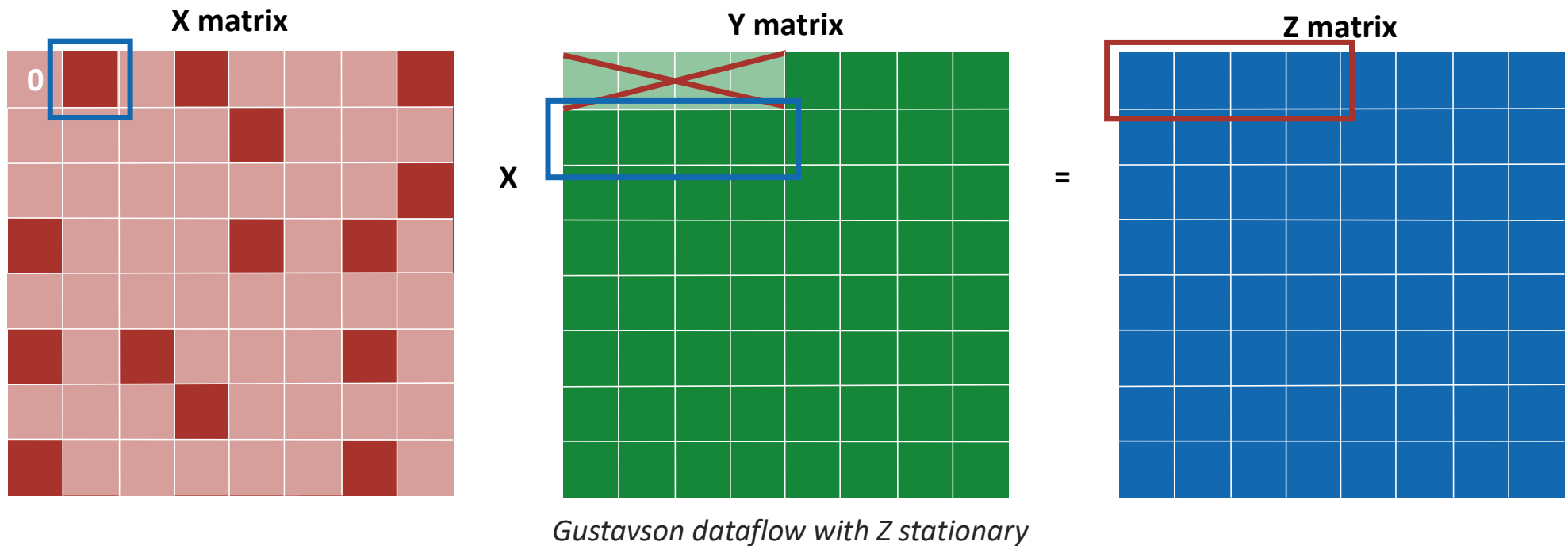
- **Assumptions:**

- MAC units = 4
- X rows = 32
- Y columns = 32
- Inner dim. = 32
- Metadata chunk = 128b
- Input data size = 32b
- Output data size = 64b
- BW = 128b

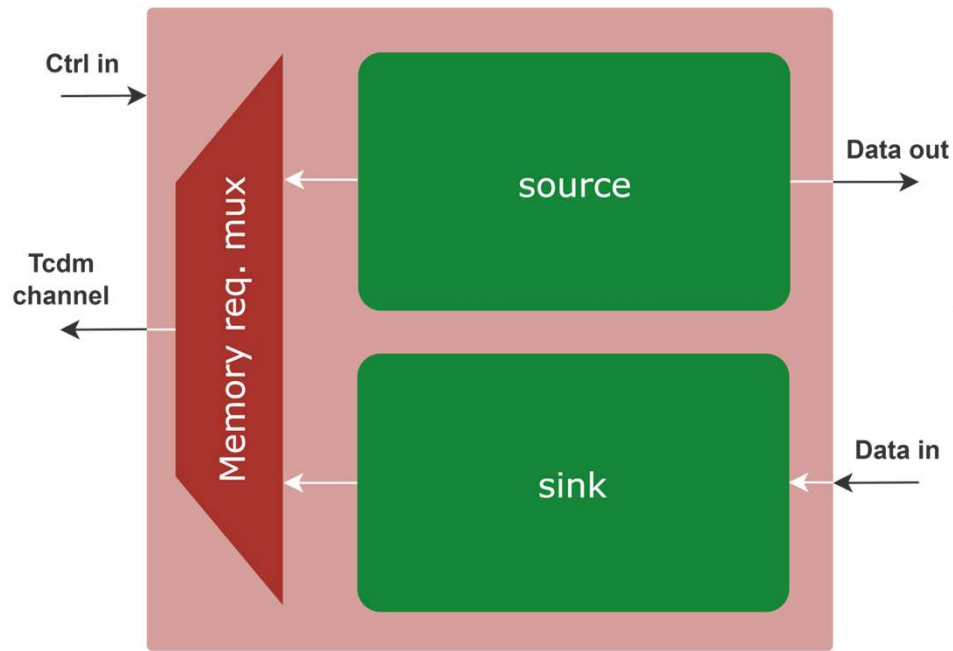


Final choice

- Overall the Gustavson dataflow with Z reuse seems to be the best
 - Never loads chunks of Z from memory
 - Reuses the metadata as much as possible

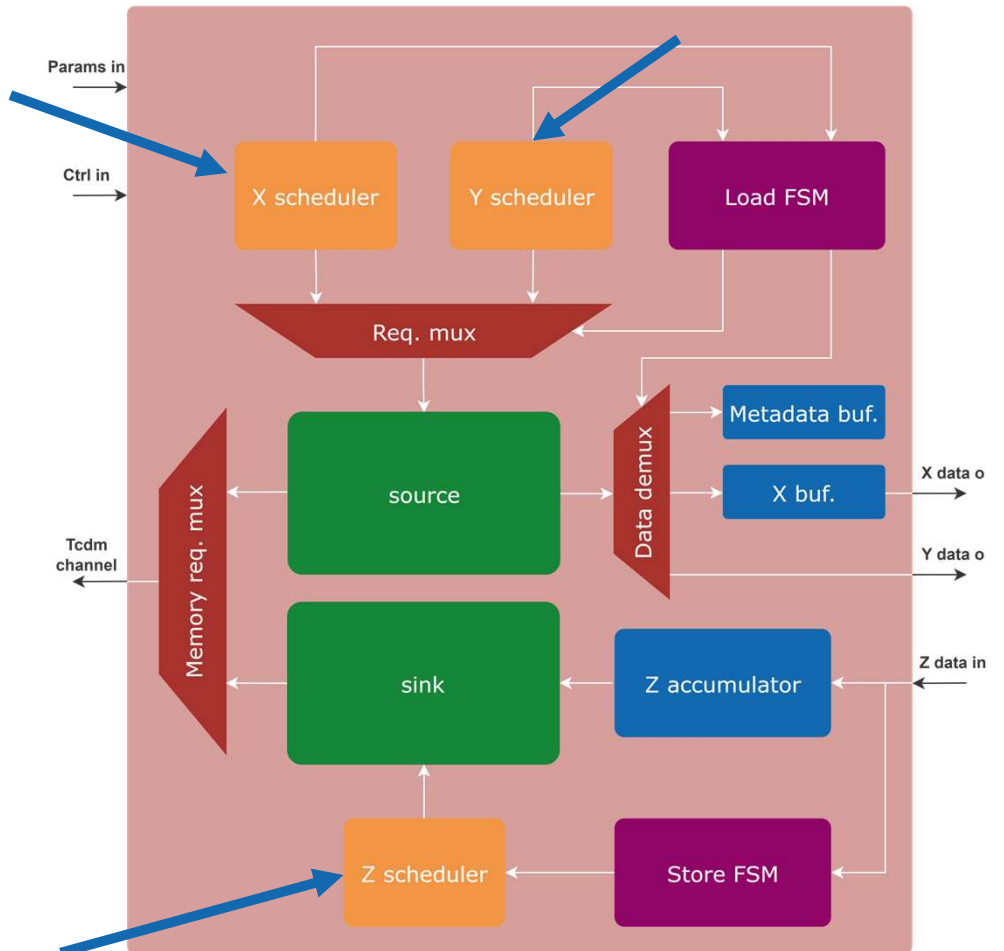


Accelerator's streamer architecture VS datamover one



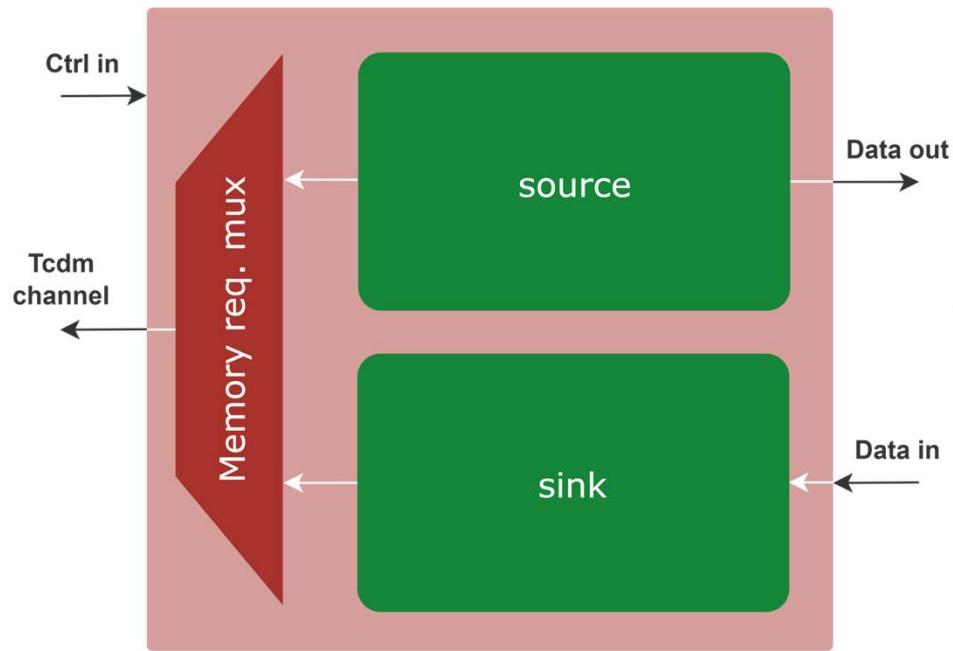
Datamover's streamer architecture

VS



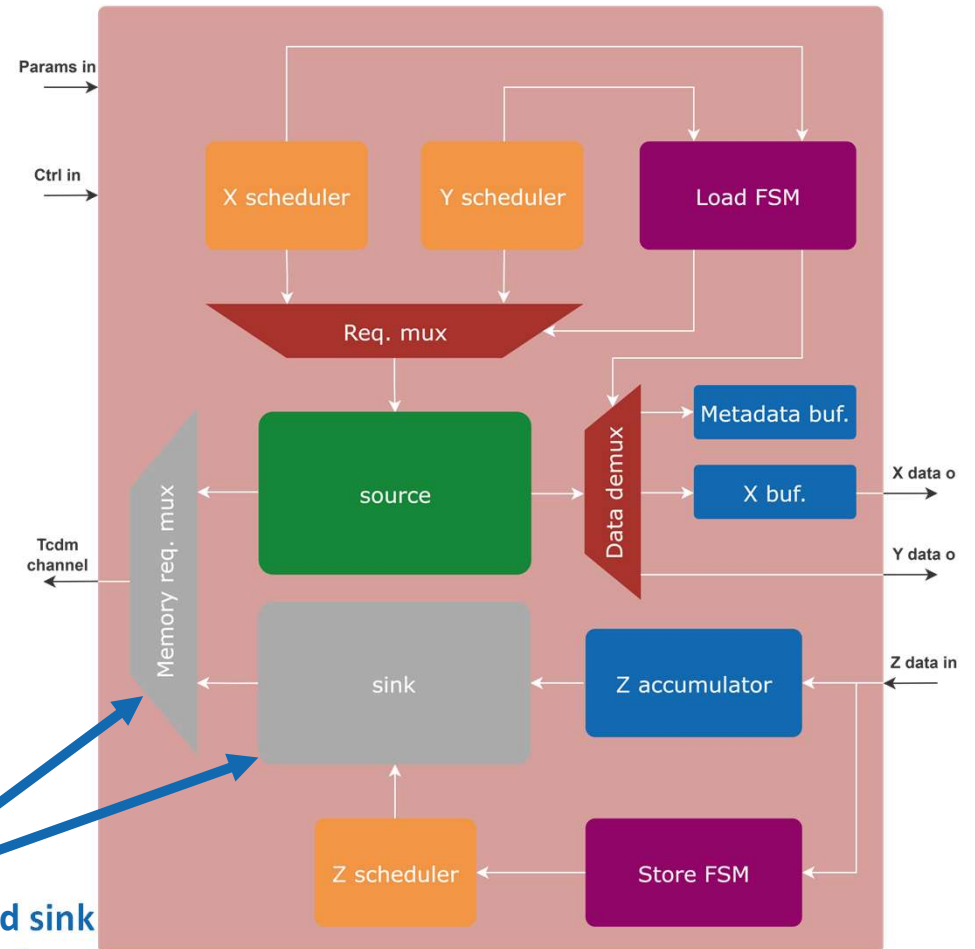
Accelerator's streamer architecture

Accelerator's streamer architecture VS datamover one



Datamover's streamer architecture

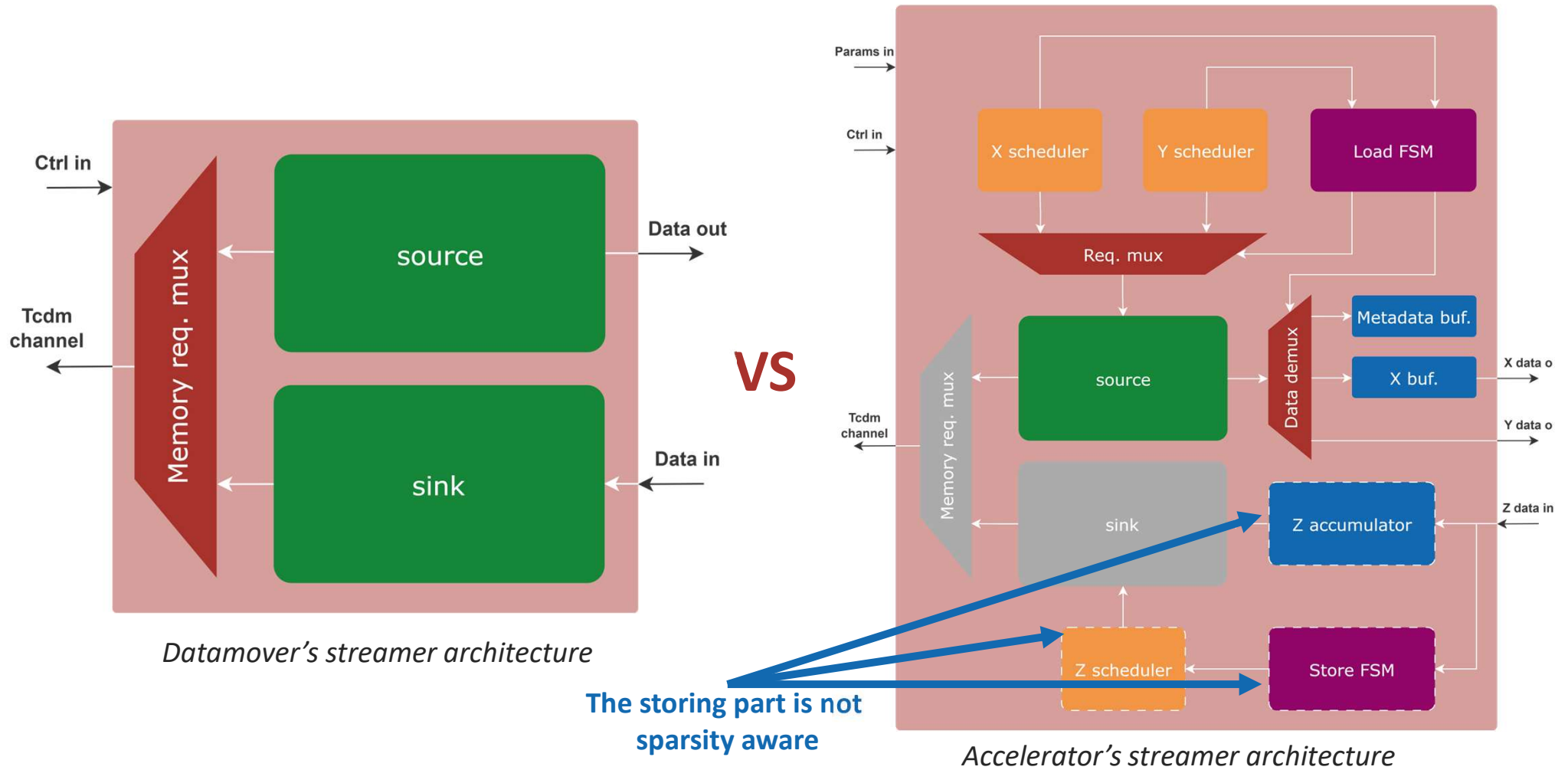
VS



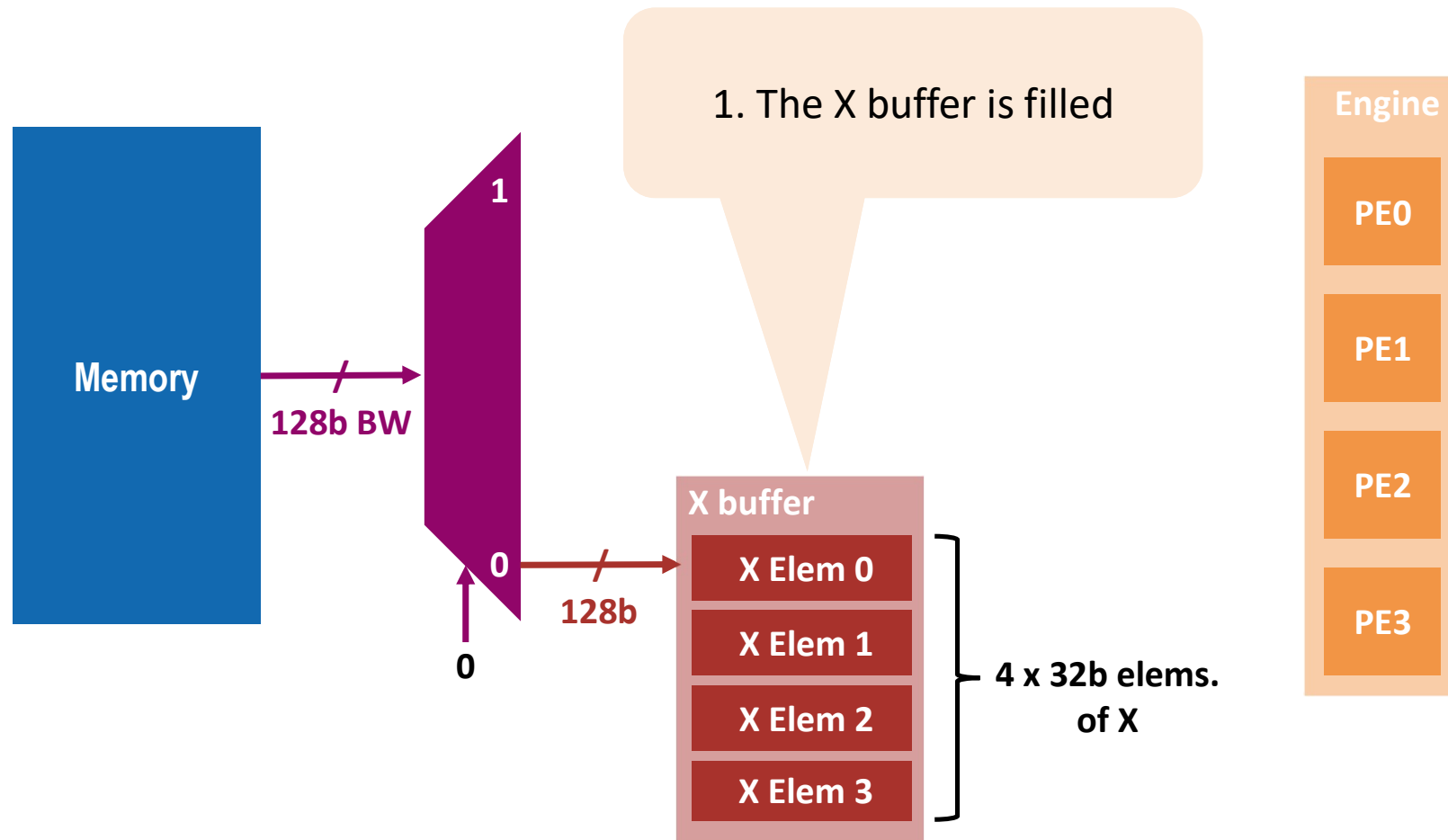
Accelerator's streamer architecture

The memory mux and sink
were not modified

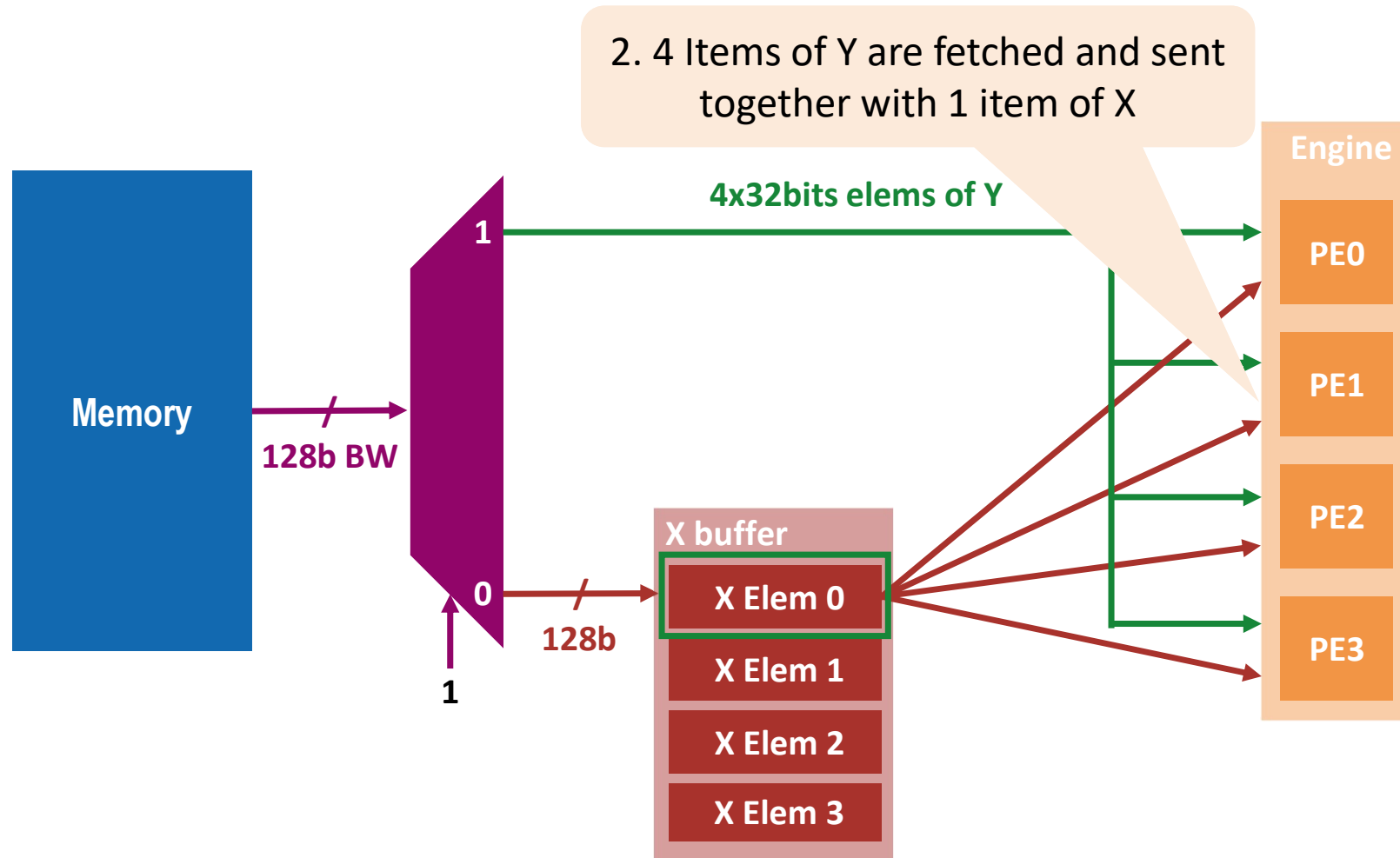
Accelerator's streamer architecture VS datamover one



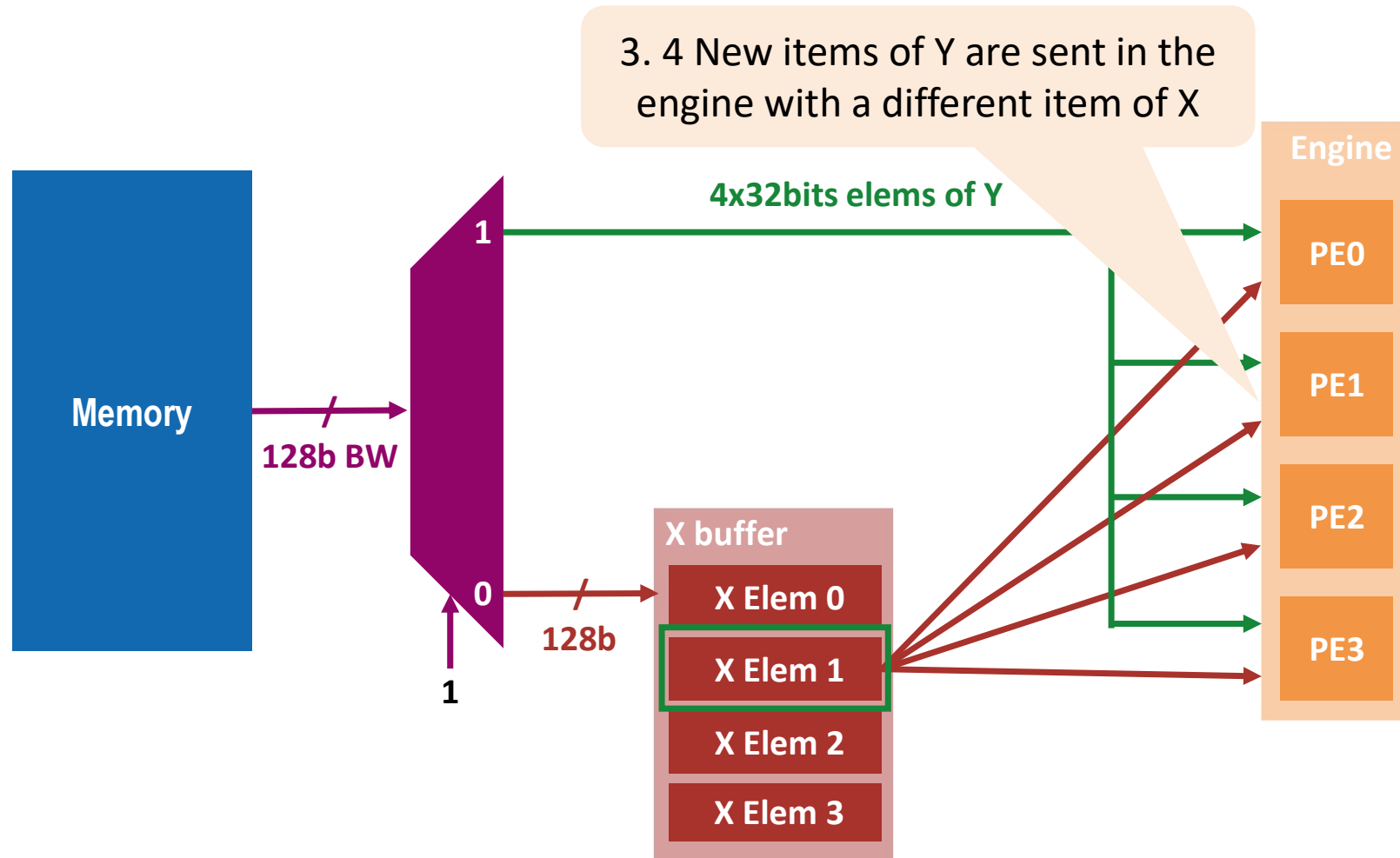
Streamer behavior –Loading part



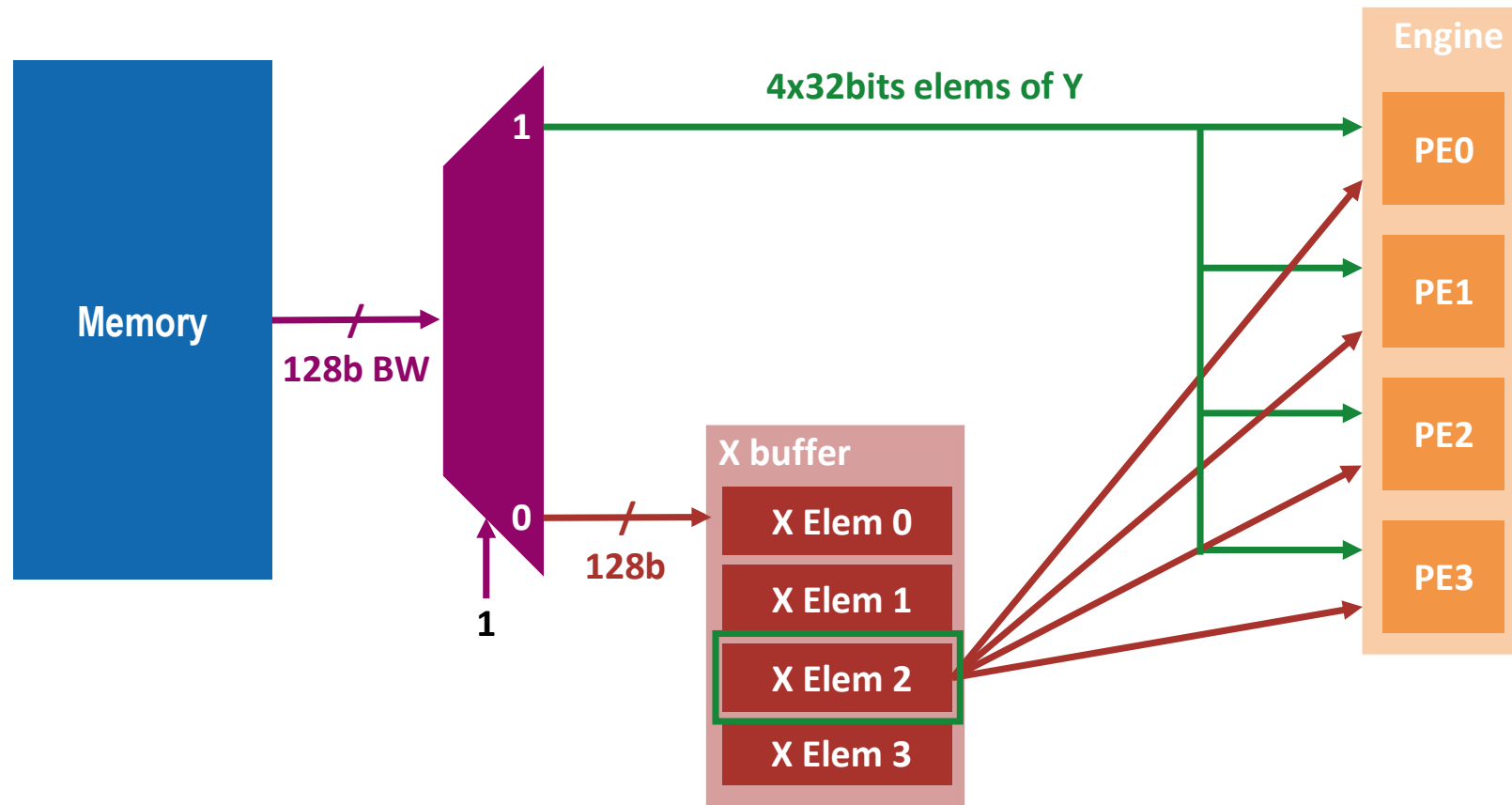
Streamer behavior –Loading part



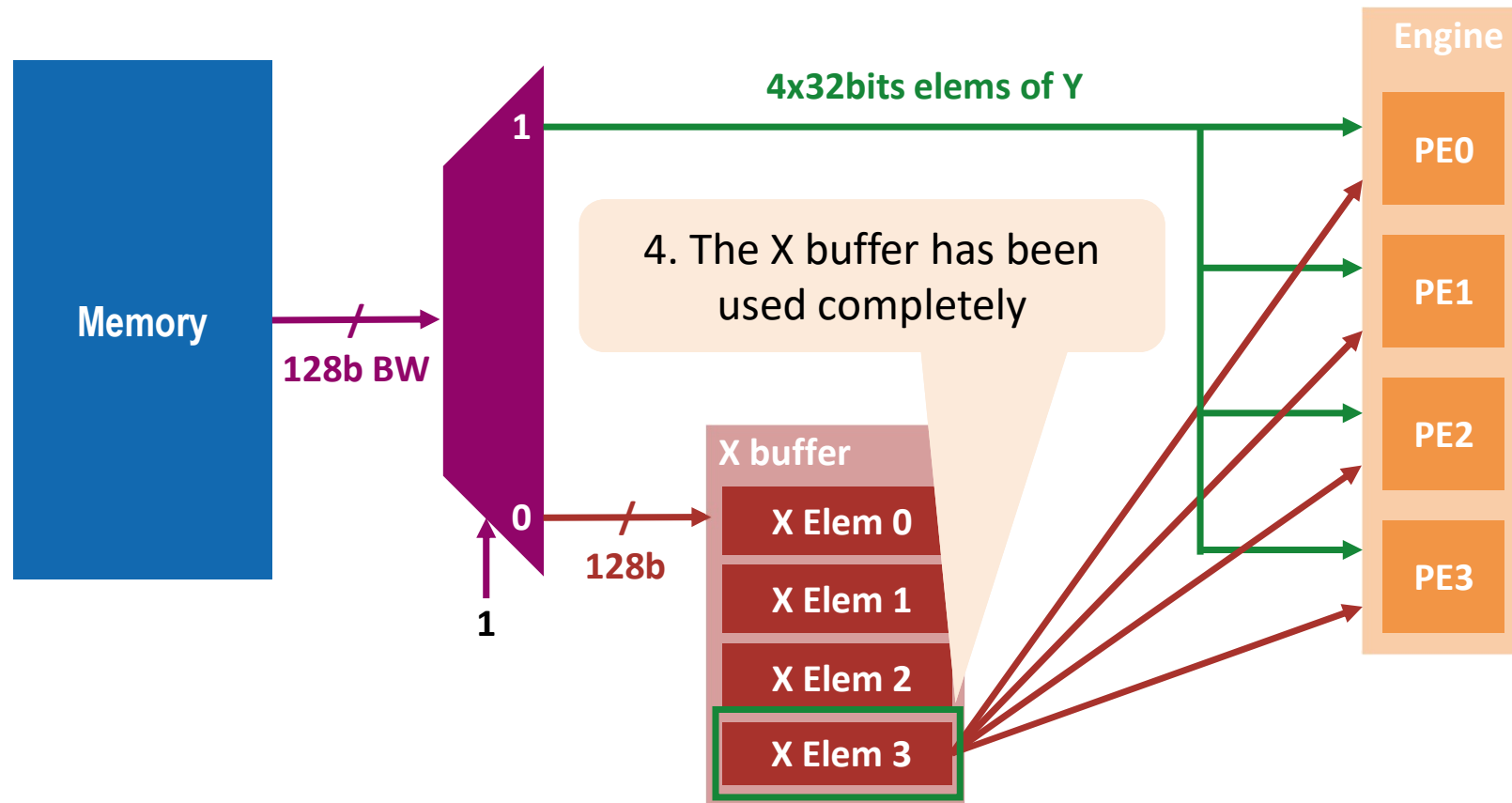
Streamer behavior –Loading part



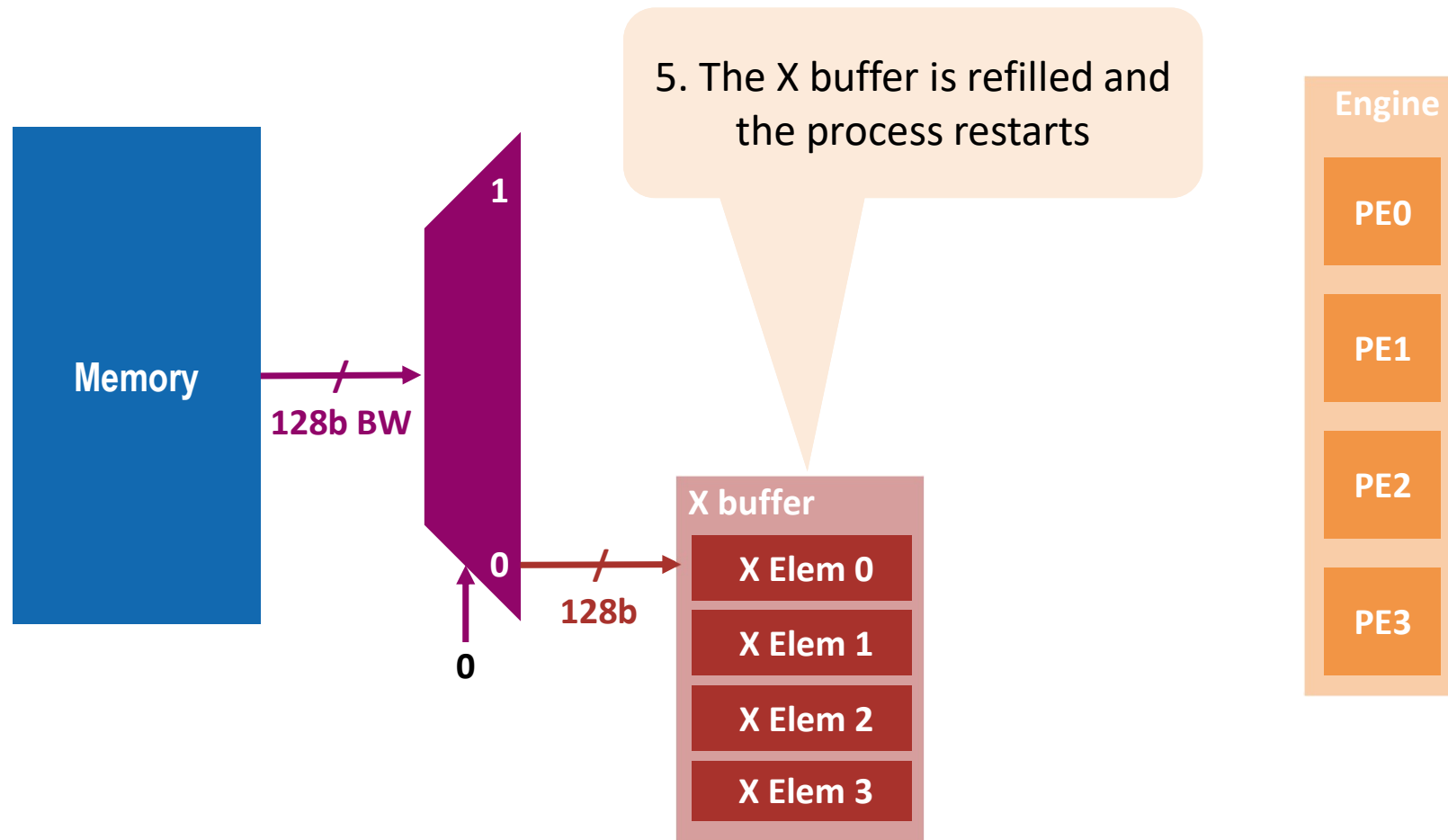
Streamer behavior –Loading part



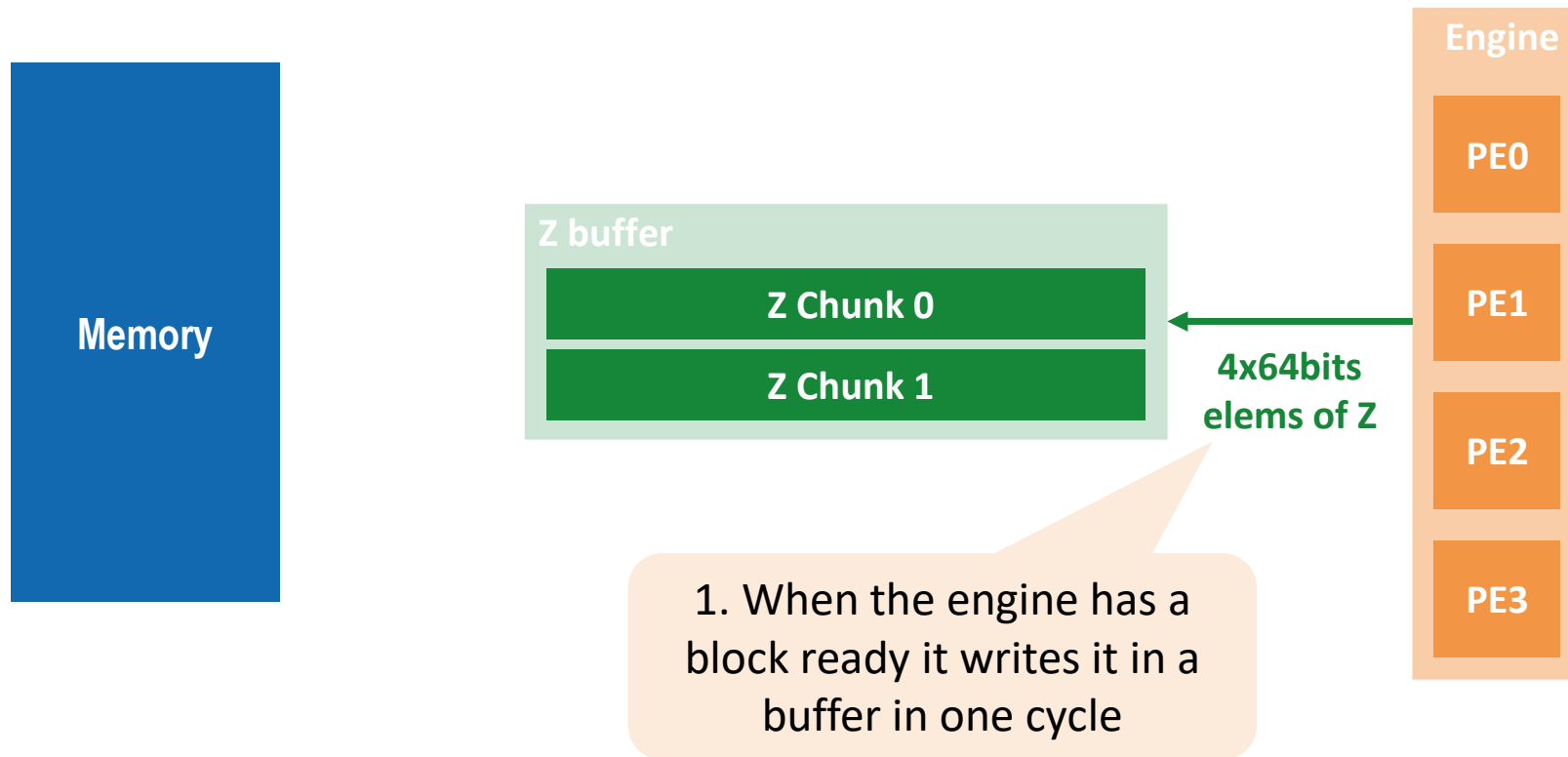
Streamer behavior –Loading part



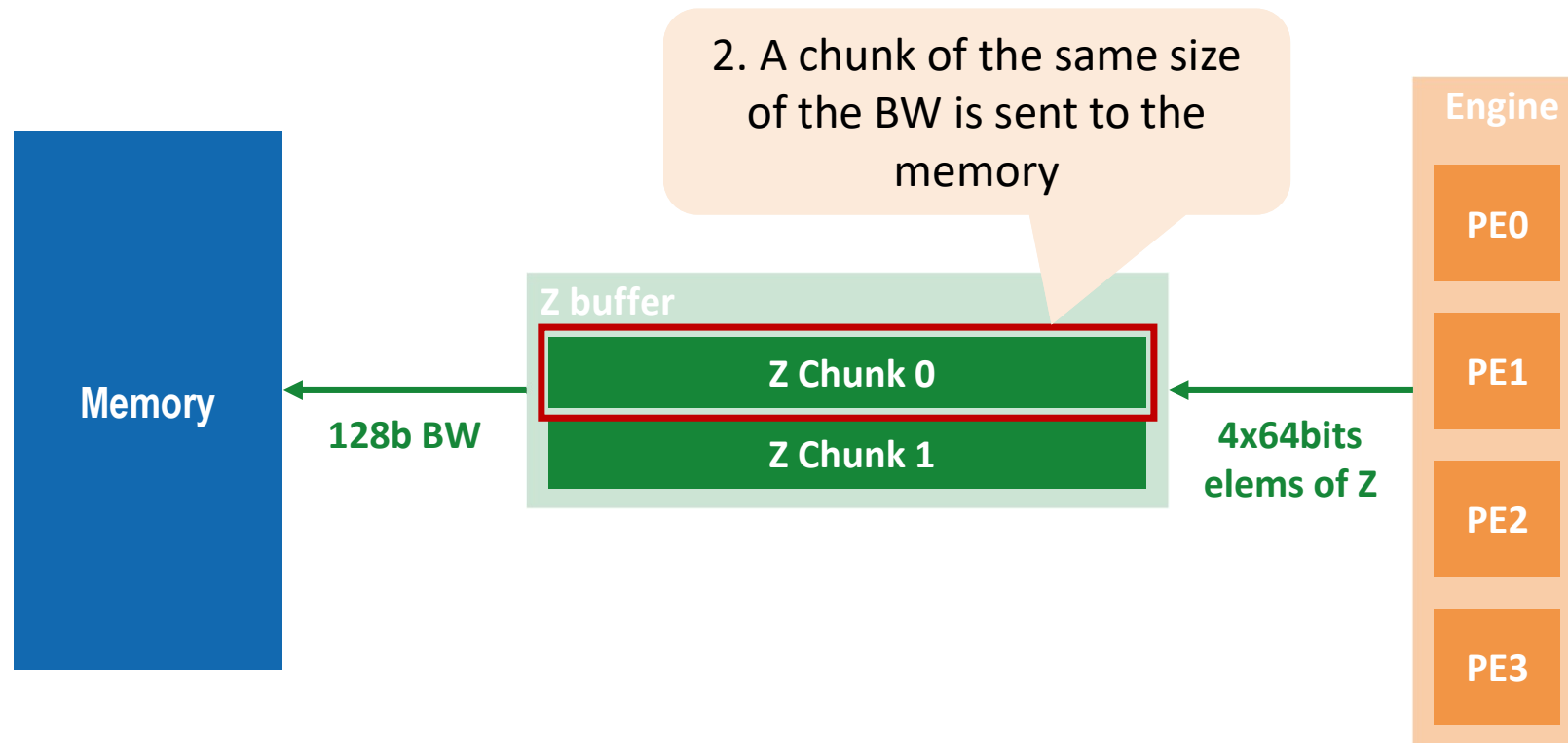
Streamer behavior –Loading part



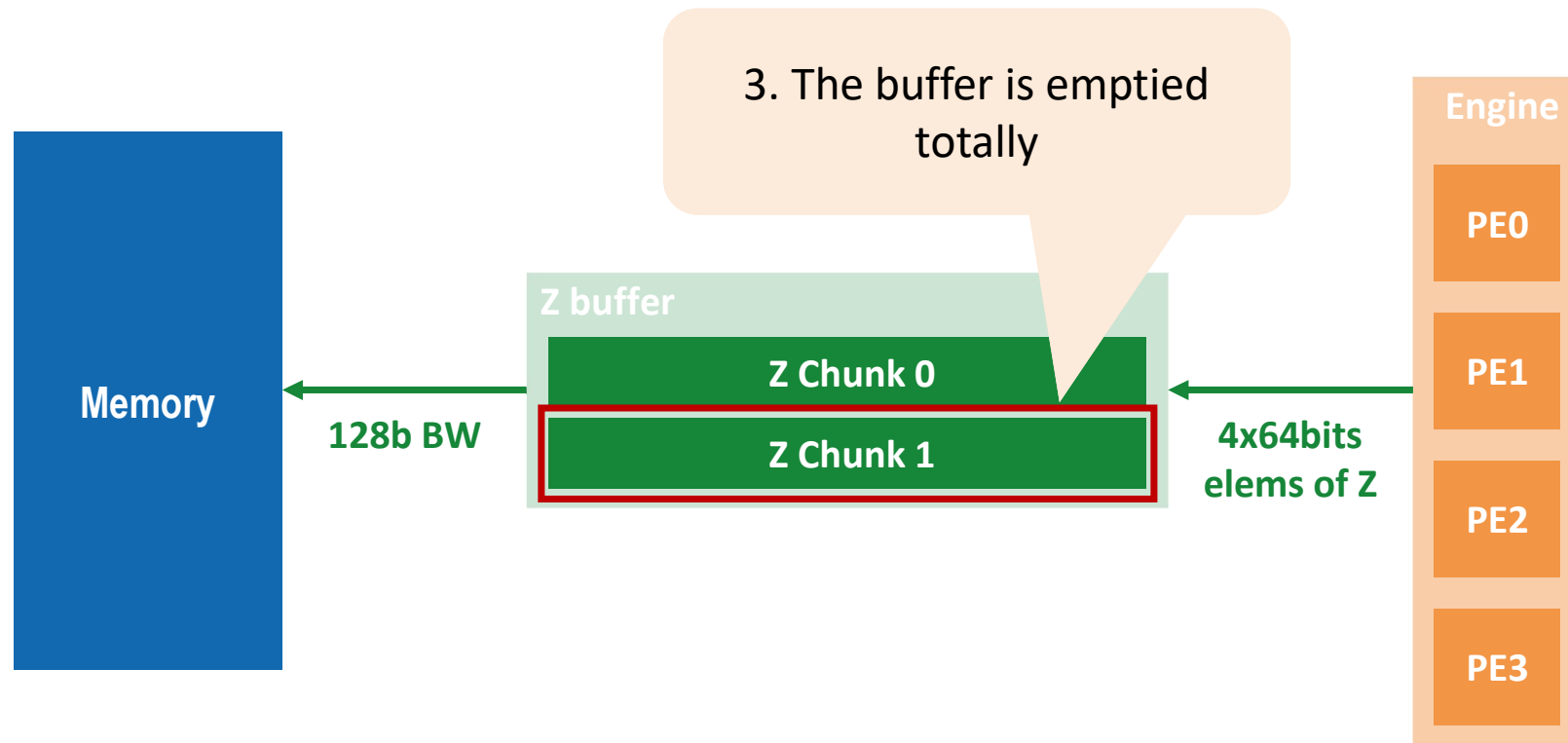
Streamer behavior –Storing part



Streamer behavior –Storing part

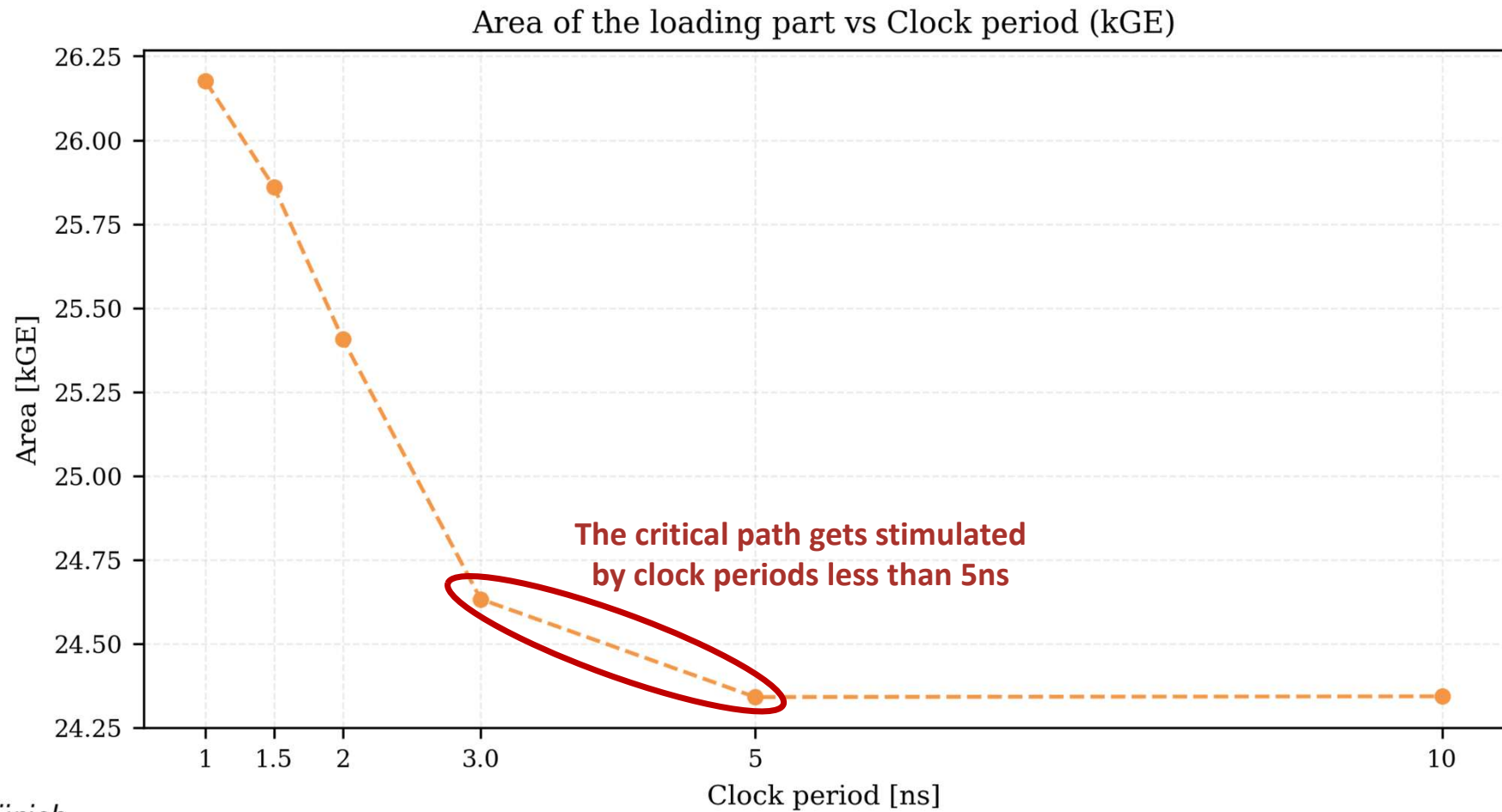


Streamer behavior –Storing part



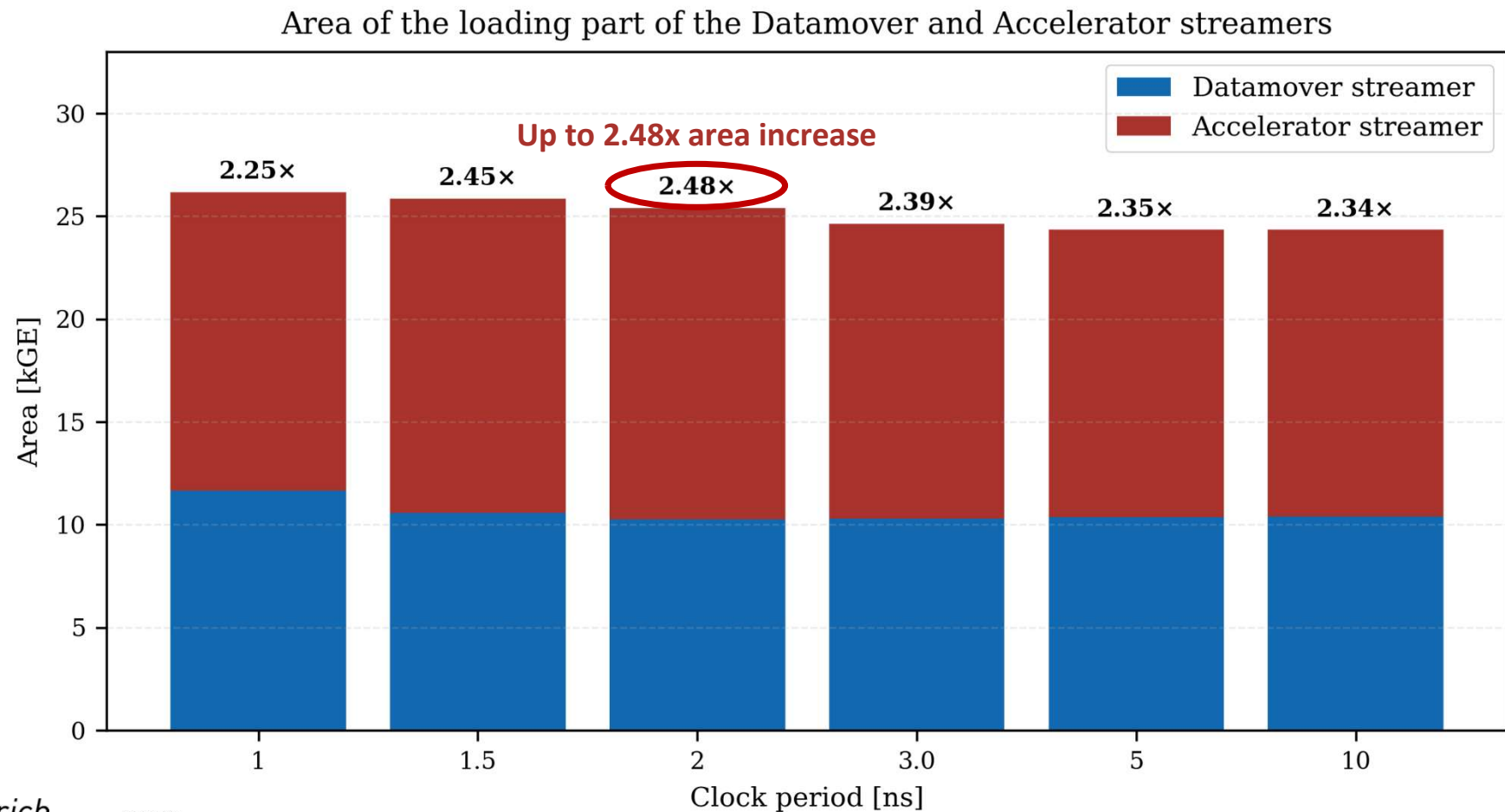
Synthesis results- AT plot

The plot reports the area considering only the loading part which was made sparsity-aware; synthesis was performed using the slow corner of the cells.



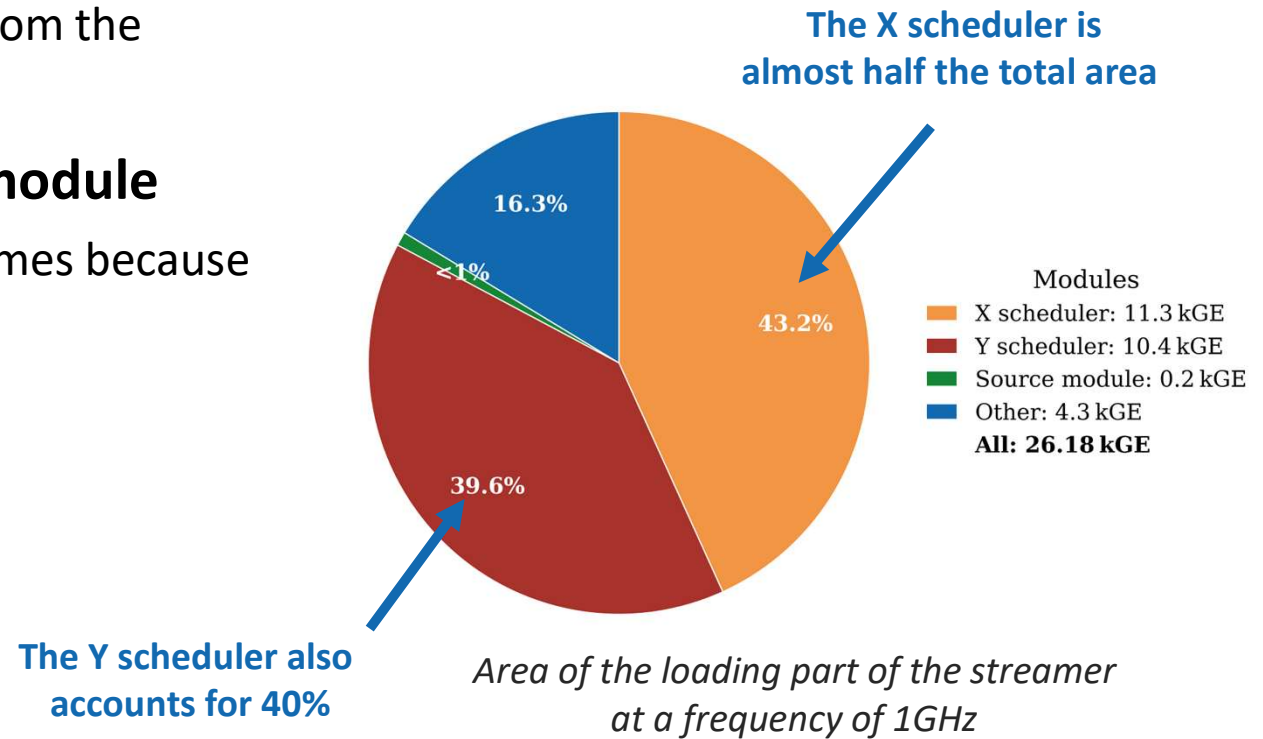
Synthesis results- Streamer VS Datamover

The plot reports the area difference only considering the loading part of the two designs, which was made sparsity-aware.



Synthesis results- Area contributions

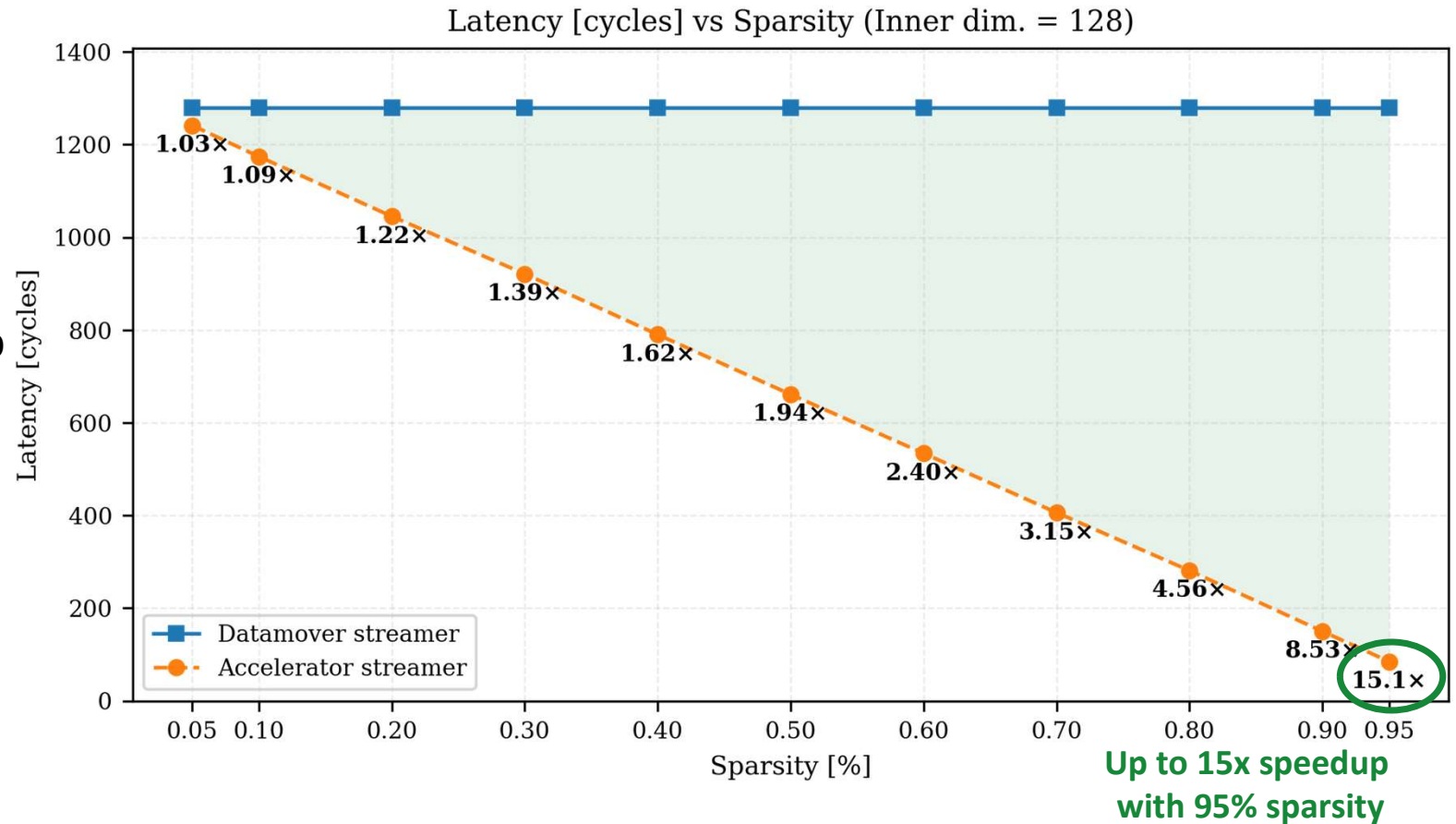
- **Adding intelligence is expensive**
 - The biggest area contributions come from the schedulers
- **Different function of the source module**
 - The source was reduced of about 10 times because it's not “smart” anymore.



RTL results –Latency VS sparsity

- **Assumptions:**

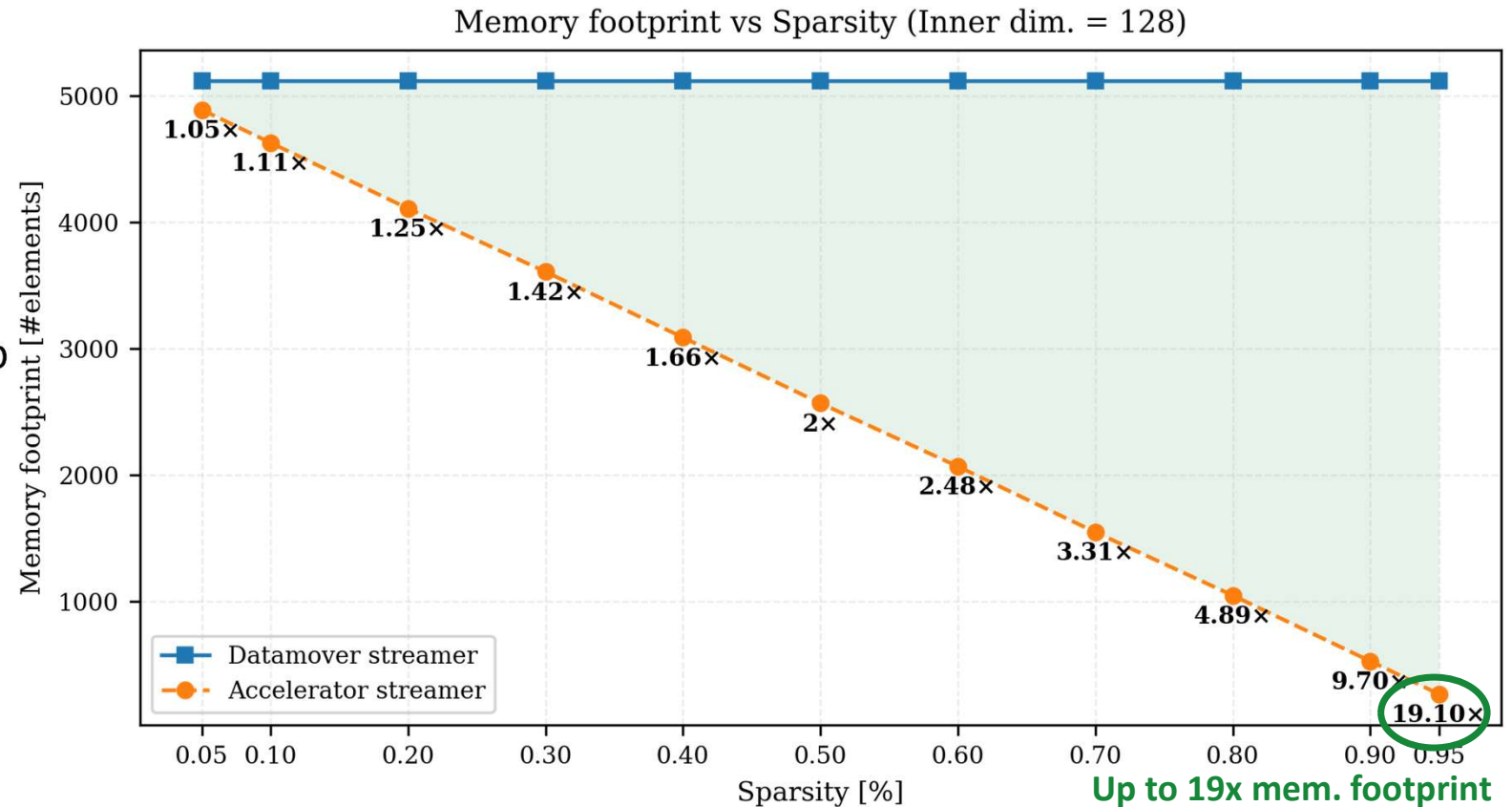
- MAC units = 4
- X rows = 2
- Y columns = 16
- Inner dim. = 128
- Metadata chunk = 128b
- Input data size = 32b
- Output data size = 64b
- BW = 128b



RTL results –Memory footprint VS sparsity

- **Assumptions:**

- MAC units = 4
- X rows = 2
- Y columns = 16
- Inner dim. = 128
- Metadata chunk = 128b
- Input data size = 32b
- Output data size = 64b
- BW = 128b



Up to 19x mem. footprint decrease with 95% sparsity

A microscopic view of a silicon chip, showing intricate circuitry and various colored regions (red, green, blue, yellow) representing different functional blocks. A large, semi-transparent blue rectangle is overlaid in the center, containing the title text.

Expanding an existing SoC

E-Bee-P-Chip -Overview

- **Context**

- **VLSI2:** two-people project work
- **Scope:** learn the full stack of an ASIC workflow

- **Goal**

- Improve an existing SoC (Croc) by adding something

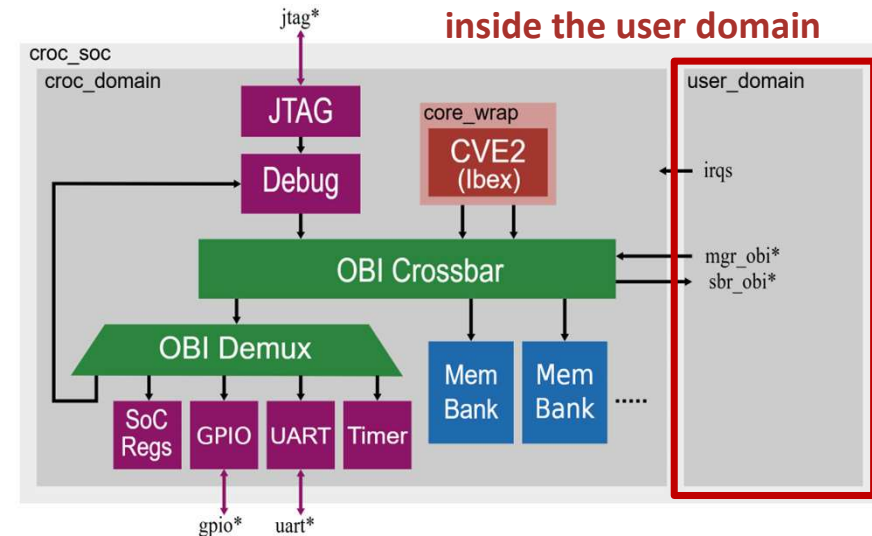
- **Workflow**

- **Implement RTL:** implemented an accelerator for data compression (EBPC), increased RAM size
- **Functional verification:** verified the functionality of the RTL with a C testbench
- **Physical implementation:** complete backend of the ASIC
- **Results:** comparison with the base SoC to measure speedup, area and power analysis



Logo of our chip

The module was implemented inside the user domain



Structure of the Croc SoC

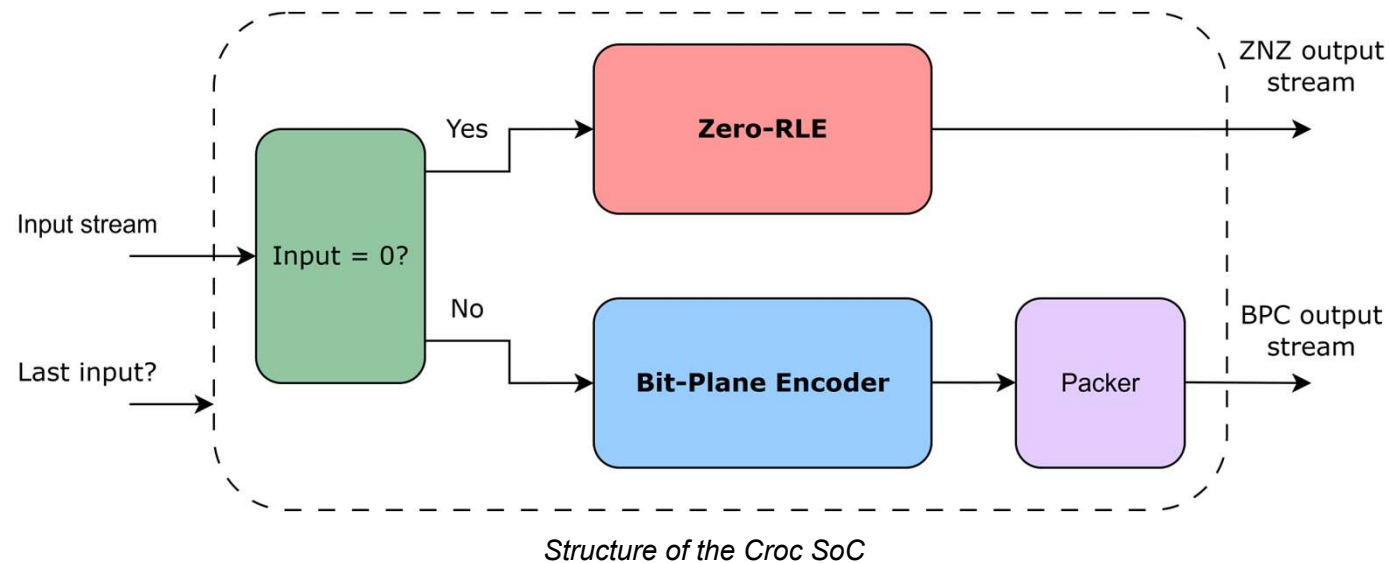
E-Bee-P-Chip -EBPC

- **Ideal conditions**

- A stream of 16-bit data items that are both sparse and correlated

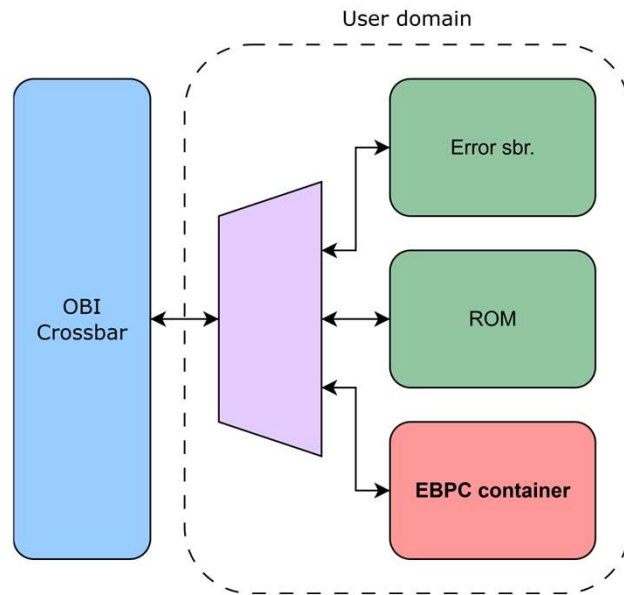
- **Two modules**

- **ZRLE**: compresses data items equal to 0
- **BPC**: compresses nonzero-correlated data

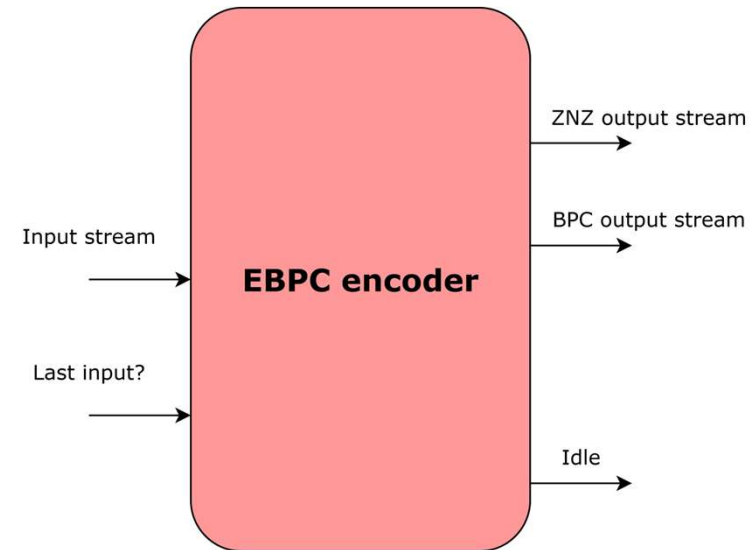


E-Bee-P-Chip –RTL implementation

- We took a ready-to use IP from this [Github Repository](#)
- Implemented inside the user domain
 - Inside a container that decodes the addresses of the OBI protocol
 - The container is connected to a sbr. Demux



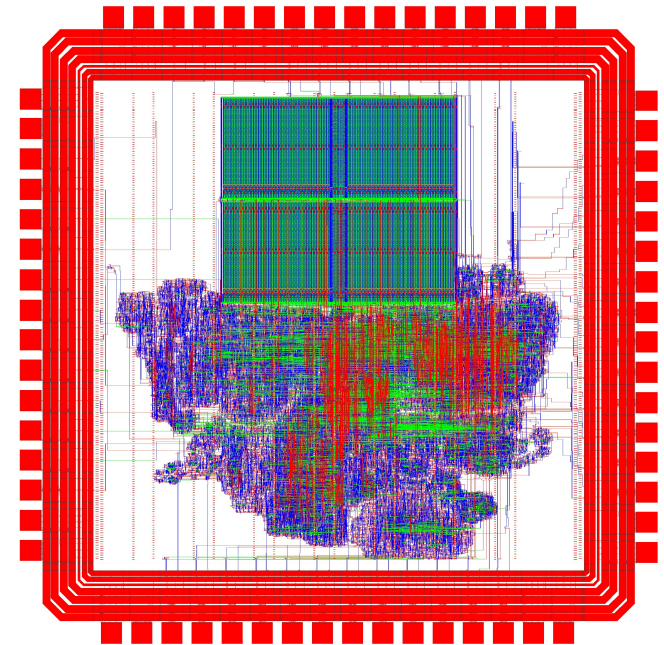
Integration of the module in the User Domain



Integration of the module in the User Domain

E-Bee-P-Chip –Backend

- **Synthesis**
 - Yosys
- **Physical backend**
 - Openroad
- **RAM banks**
 - From 2x2KB to 2x8KB
- **Clock frequency**
 - From 72MHz to 70MHz



Final layout

	Active area [mm ²]	Core area [mm ²]	Core utilization
Croc	0.78	1.76	44.4%
E-Bee-P-Chip	1.25	2.51	49.6%

Area results

	Power [mW]
Croc	51.0
E-Bee-P-Chip	51.7

Power results

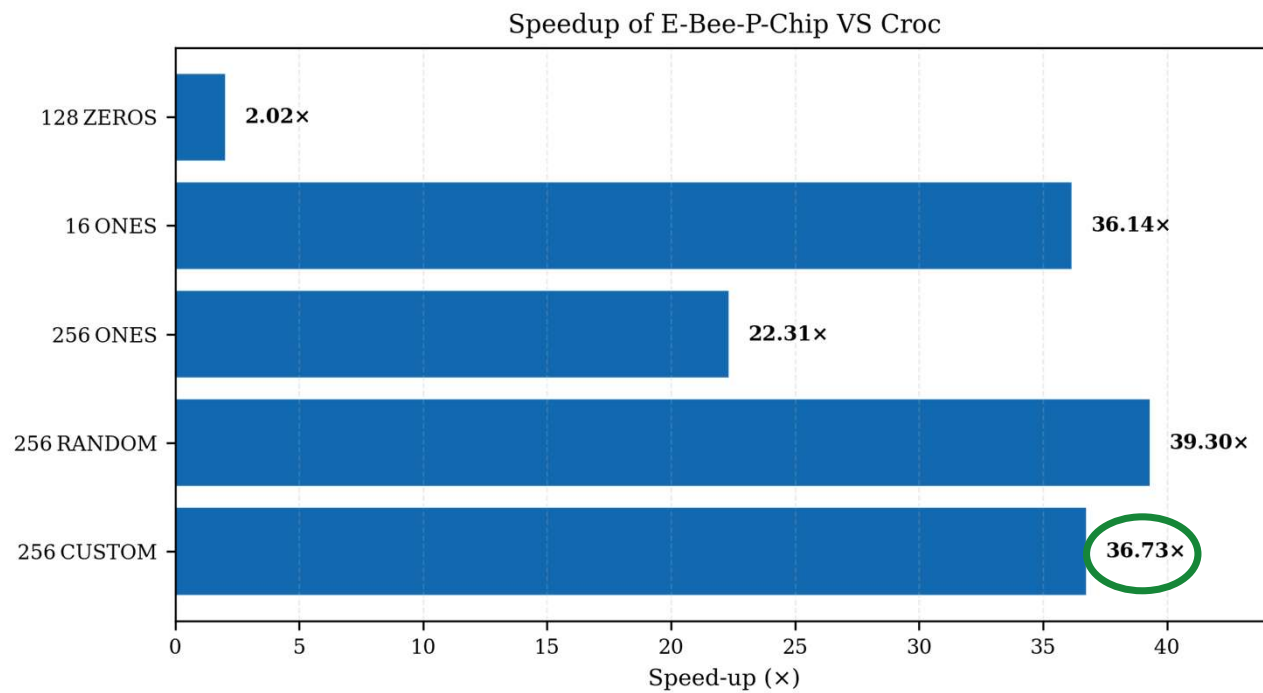
E-Bee-P-Chip – Benchmarks

- **Method**

- C testbench that uses our accelerator VS C testbench that only uses core

- **Different use cases**

- Custom is made of 256 16-bit sparse and correlated data items



Up to 37x speedup
for the realistic case